The Journal of Systems and Software 223 (2025) 112373

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Architectural tactics to achieve quality attributes of machine-learning-enabled systems: a systematic literature review☆

Vladislav Indykov [a,*], Daniel Strüber [a,b], Rebekka Wohlrab [a,c]

[a] University of Gothenburg and Chalmers University of Technology, Gothenburg, Sweden
[b] Radboud University, Nijmegen, Netherlands
[c] Carnegie Mellon University, Pittsburgh, USA

ARTICLE INFO

Dataset link: https://doi.org/10.6084/m9.figshare.25673643.v1

Keywords:
Machine learning
Software architecture
Software quality
Quality attributes

ABSTRACT

Machine-learning-enabled systems are becoming increasingly common in different industries. Due to the impact of uncertainty and the pronounced role of data, ensuring the quality of such systems requires consideration of several unique characteristics in addition to traditional ones. This range of quality attributes can be achieved by the implementation of specific architectural tactics. Such architectural decisions affect the further functioning of the system and its compliance with business goals. Architectural decisions have to be made with attention to possible quality trade-offs to prevent the cost of mitigating unintended side effects. A related work analysis revealed the need for a thorough study of existing architectural decisions and their impact on various quality attributes in the context of machine-learning-enabled systems. In this paper, to address this goal, we present comprehensive research on the quality of such systems, architectural tactics, and their possible quality consequences. Based on a systematic literature review of 206 primary sources, we identified 11 common quality attributes, and 16 relevant architectural tactics together along with 85 potential quality trade-offs. Our results systematize existing research in building architectures of ML-enabled systems. They can be used by software architects and researchers at the system design stage to estimate the possible consequences of decisions made.

Contents

1. Introduction 2
2. Background 3
   2.1. Context 3
   2.2. Related work 3
3. Methodology 3
   3.1. Review questions 3
   3.2. Inclusion and exclusion criteria 4
   3.3. Data sources 4
   3.4. Data collection 4
   3.5. Data synthesis 5
   3.6. Results verification 6
4. Results 6
   4.1. RQ1: Identification of common quality model 6
      4.1.1. Studies based on interviews and questionnaires 6
      4.1.2. Studies based on expert assessments 6
      4.1.3. Studies based on the other methodologies 7
      4.1.4. Synthesized quality model 7
      4.1.5. Verification of the quality model 8
   4.2. RQ2: Architectural tactics 8

☆ Editor: Alexander Serebrenik.

* Corresponding author.
E-mail address: indykov@chalmers.se (V. Indykov).
URL: https://euphort.se/ (V. Indykov).

https://doi.org/10.1016/j.jss.2025.112373
Received 17 July 2024; Received in revised form 5 February 2025; Accepted 5 February 2025
Available online 13 February 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

4.2.1. Tactics associated with resource efficiency ... 8
4.2.2. Tactic associated with usability ... 9
4.2.3. Tactics associated with reliability ... 9
4.2.4. Tactics associated with security ... 9
4.2.5. Tactics associated with maintainability ... 10
4.2.6. Tactics associated with portability ... 10
4.2.7. Tactics associated with explainability ... 10
4.2.8. Tactics associated with system accuracy ... 11
4.2.9. Tactic associated with fairness ... 11
4.2.10. Tactics associated with data quality ... 11
4.2.11. Definitions of architectural tactics ... 11
4.2.12. Verification of the architectural tactics ... 12
4.3. RQ3: Trade-off analysis ... 12
4.3.1. AT1: Distributed learning ... 12
4.3.2. AT2: Automated data reduction ... 12
4.3.3. AT3: Federated learning ... 13
4.3.4. AT4: Human-in-the-Loop (HitL) ... 13
4.3.5. AT5: Automated data versioning ... 13
4.3.6. AT6: Intrusion detection ... 13
4.3.7. AT7: Automated data encryption ... 13
4.3.8. AT8: Containerization ... 13
4.3.9. AT9: Componentization ... 13
4.3.10. AT10: Local interpretable models (LIME) ... 14
4.3.11. AT11: Rule-based models ... 14
4.3.12. AT12: Automated hyperparameter tuning ... 14
4.3.13. AT13: Automated algorithm selection ... 14
4.3.14. AT14: Automated bias mitigation ... 14
4.3.15. AT15: Automated data preprocessing ... 14
4.3.16. AT16: Automated data profiling ... 14
4.3.17. Verification of the trade-off matrix ... 14
5. Discussion ... 15
5.1. Observations regarding quality attributes ... 15
5.2. Comparison to ISO standards ... 15
5.3. Observations regarding trade-offs ... 15
5.4. Threats to validity ... 15
5.5. Implications for practitioners ... 16
5.6. Implications for researchers ... 16
6. Conclusion ... 16
CRediT authorship contribution statement ... 16
Declaration of competing interest ... 17
Acknowledgments ... 17
Data availability ... 17
References ... 17

1. Introduction

Machine-learning-enabled (ML-enabled) systems (Hulten, 2018) are currently in high demand among various spheres. The design, development, and implementation of such systems are widespread now since ML technologies allow organizations to reach results that are difficult to achieve through traditional solutions. Machine learning systems typically work with large volumes of data, adapt, learn, search for, and process complex correlations. The development of AI-based systems is an extremely relevant strategy for the world's largest vendors: Meta is implementing ML components for content moderation and feed personalization, Microsoft is focused on developing the AI companion called "Microsoft Copilot", the use of large language models is conquering new frontiers. However, designing such systems remains a non-trivial and non-standardized task due to the lack of detailed system-level guidelines and instructions for constructing appropriate architectures with a consideration of system specifics.

The construction of ML-based software architecture starts with the collection of requirements, particularly, non-functional ones, also known as *quality attributes*. They must be considered at the stage of architectural design to align the system with the intended goals. As Monson stated: "*You don't drive the architecture, the requirements do. You do your best to serve their needs*" (Monson-Haefel, 2009).

There are several detailed specifications and standards (*e.g.*, ISO/IEC 25010 International Organization for Standardization, 2011a and ISO/IEC 45010 International Organization for Standardization, 2011b) for traditional software that makes it possible to predetermine the fundamental quality attributes of the designed system without conducting any deep research. Some of their quality characteristics can be adapted, updated, and extended to directly meet the needs of ML-based software due to its unique characteristics compared to traditional ones. Specifically, ML-enabled systems operate in environments of high uncertainty and depend on the quality and quantity of data used for model training, validation, and testing. This fact and its relevance were confirmed by the ISO/IEC 25059 (International Organization for Standardization, 2023) issued in June 2023, which adjusted some of the existing qualities from ISO/IEC 25010 to ML contexts and additionally considered several ML-specific aspects (*e.g.*, ethics, transparency). While this new standard presents an important initiative for addressing the specific quality aspects of ML-enabled systems, it has not been investigated to which extent it characterizes the relevant aspects of this domain exhaustively. Such an investigation could be supported by a systematic study, as we perform in this work.

Quality attributes can be achieved by architectural and non-architectural tactics. Non-architectural ways of achieving quality are based on organizational non-technical management and on technical decisions that do not affect software architecture. Such decisions are

too dependent on the system specifics and are out of our scope. Architectural tactics, on the contrary, are general design decisions. They are designed to improve one specific target quality attribute. In practice, it is very common that architectural tactics entail unanticipated tradeoffs on other quality attributes. To raise awareness of the consequences that come with a selection of architectural tactics, it is important to make these tradeoffs explicit. This is one of the contributions of this paper and will enable architects to more deliberately select architectural tactics for ML-enabled systems in the future.

For example, there is an architectural tactic to implement a real-time data monitoring module. In the context of ML-enabled systems, by implementing this architectural tactic, the operator gets an opportunity to monitor all the data used for model training, testing, and validation as well as dynamic input data. Such a decision can increase fairness, reliability, maintainability, accuracy, and security. However, the main trade-off after the implementation of this tactic appears in terms of resource efficiency when operating with big data (Wang and Miao, 2022), Hassan et al. (2019), Wanganoo and Shukla (2020) .

In this paper, we give a comprehensive picture of architectural tactics for the engineering of ML-enabled systems along with quality attributes affected by them. We report on the results of a systematic literature review, in which we extracted information from 206 primary sources about these aspects. Specifically, we made the following contributions:

1. We propose a quality model for ML-enabled systems, focused on the most commonly reported quality attributes in the literature, and compare it to the relevant standards of ISO/IEC 25010 (International Organization for Standardization, 2011a) and ISO/IEC 25059 (International Organization for Standardization, 2023).
2. We present a range of architectural tactics that can help achieve identified common quality attributes.
3. We present an analysis of the quality trade-offs of the identified architectural tactics, summarized as an impact matrix.

This paper is accompanied by a supplementary artifact[1] which contains search queries and data extraction sheets.

The rest of this paper is structured as follows: Section 2 discusses related work and introduces the used terminology. Section 3 describes our research methodology. Section 4 presents our results, including the identification and analysis of quality attributes, architectural tactics, and quality trade-offs. Section 5 discusses implications concerning specific attributes, other quality standards, and threats to validity. Section 6 concludes and outlines future work. Section 7 provides a data availability statement. Section 8 presents the acknowledgments.

2. Background

2.1. Context

A *quality attribute (QA)* is a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders (Bass et al., 2003). In ISO/IEC 9126-1:2001, quality attributes are described as a *"checklist to determine software quality"* (International Organization for Standardization, 2001). According to Lundberg et al. (1999), the quality attributes should guide the design of the software architecture. While stakeholders, usage contexts and, therefore, relevant quality attributes differ from one system to another, one can identify the most widespread quality attributes applied to systems of different natures. In the context of this work, we call them the *"common quality attributes" (CQAs)*.

Quality attributes are related to the term "architecturally significant requirement(s)". However, the latter is entirely specialized to a particular system, based on the needs of certain stakeholders, technical capabilities, internal regulations, etc. "*In gathering [architecturally significant requirements], we should be mindful of the business goals of the organization*" (Bass et al., 2003). In this paper, we seek to generalize existing experience, putting the specifics of individual systems aside.

An *architectural tactic (AT)* is a "*technique an architect can use to achieve the required quality attributes*" (Bass et al., 2003). By definition, the connection between tactic and certain *quality attributes* is implied. However, our study goes further and analyzes the impact of its influence on all identified common quality attributes. Balance or compromises between them are called *quality trade-offs*.

2.2. Related work

The study of software quality for ML-enabled systems is an in-demand topic among researchers and practitioners (Serban and Visser, 2022), Santhanam (2020). Despite the relatively small number of studies published at the time of writing the current paper, a steady positive trend in this domain was noted. The space for interpreting the quality of AI systems has only been partially explored and a conclusive view is yet to form, which is proved by the emergence of different quality models based on industrial experience (Siebert et al., 2022; Kuwajima et al., 2020; Gezici and Tarhan, 2022). Such studies work with non-functional requirements relevant to a certain system and most often receive them from domain experts. The generalizability of such models can be debatable due to context dependence. Their systematization and the identification of the most common quality attributes is a way to build a more generalized picture based on real examples. Such a strategy supports a collection of the most recent materials and makes current research more independent from external inputs.

There are also plenty of review papers on architectural issues in the context of AI-based systems (Franch et al., 2022; Bhat et al., 2020; Muccini and Vaidhyanathan, 2021). These papers explore a collection of existing architectural design decisions without a clear reference to system qualities or with a focus on the impact of decisions on individual quality attributes and their metrics in isolation from the overall quality picture of the system. As a result, possible trade-offs often remain unnoticed. In contrast with such studies, we strive to investigate the effects of architectural tactics (ATs) on all the identified quality attributes to provide insights at the architectural level.

3. Methodology

The methodology of *systematic literature review (SLR)* allowed us to work with a large amount of scientific information, find common approaches to different systems, and effectively extract information from different sources. Such opportunities suit the research in the chosen domain. We decided to perform an SLR according to Kitchenham's guidelines (Kitchenham and Charters, 2007) as we found them most detailed and highly applicable to the current study of software architectures.

3.1. Review questions

To achieve the research objectives, three fundamental review questions (RQs) were identified.

RQ1: What are the most frequently reported quality attributes for ML-enabled systems? This question aims to identify the most often emphasized QAs in scientific literature.

RQ2: What architectural tactics have been reported to be effective for ML-enabled systems? This question aims to identify ATs to achieve quality attributes defined in RQ1. If the quality attribute cannot be satisfied by any AT, then it is out of scope for RQ2 and RQ3.

RQ3: For each architectural tactic, what is the reported impact on all the identified quality attributes? This question aims to identify quality trade-offs when ATs defined in RQ2 are implemented.

[1] Supplementary Artifact: https://figshare.com/s/57b4fa3f53caecd4a5b1

3.2. Inclusion and exclusion criteria

Only scientific literature was analyzed in this work, leaving gray literature outside the scope of this study. We used the following *inclusion criteria*:

1. Research scientific papers containing lists of QAs for specific or general ML-enabled system(s);
2. Research and review scientific papers with the description of ATs and their influence on the QA(s) of specific or general ML-enabled system(s).

We used the following *exclusion criteria*:

1. Gray literature;
2. Scientific papers about QAs of non-ML-enabled systems;
3. Scientific papers about ATs in non-ML-enabled systems;
4. Scientific papers about applying ML to address software quality concerns of non-ML-enabled systems;
5. Scientific papers about applying ML to address architectural concerns of non-ML-enabled systems;
6. Exclusively for RQ1: secondary research (literature reviews).

For our investigation of RQ1, in which we counted the number of occurrences of specific quality attributes in the literature, we deliberately excluded secondary studies. This is to avoid bias that would arise if the same primary study and its contained quality attributes are considered several times: through considered secondary studies and through our own data collection. For RQ2 and RQ3 we found it reasonable to leave secondary research included to expand the search and collect architectural tactics as much as possible. The limitation on gray literature is justified by the availability of a sufficient amount of "white" literature for the current study.

3.3. Data sources

Our search procedure was targeted to enable precise investigation of the identified research questions. To this end, we selected appropriate digital libraries and determined a suitable publication time frame.

Literature databases. To build up a high-quality review, only publications from journals and conference proceedings indexed by at least one globally significant citation database (*e.g.*, Scopus, Web of Science, etc.) were analyzed. The five most popular and largest online digital libraries were the sources for this research: IEEE Xplore (ieeexplore.ieee.org), ACM Digital Library (dl.acm.org), Springer (springerlink.com), Elsevier (sciencedirect.com), Wiley (onlinelibrary.wiley.com).

Time frame. Since this study seeks to explore the most relevant experience in the field of ML-enabled system design, we decided to limit the number of papers with the earliest date of publication of 2011. This decision was made also in connection with the release of the most recent version of the ISO/IEC 25010, which dates to 2011 (International Organization for Standardization, 2011a). This standard is important for the study since this research seeks, in some sense, to clarify the list of quality attributes from it with a consideration of the ML-enabled specifics and recent research experience. Thus, this review is based on the papers from 2011 to 2024 (the year of writing).

3.4. Data collection

Each review question implies its own objective. The architectural tactics are often not mentioned in works related to software quality and the trade-offs are often not considered in the works on a certain architectural tactic. Thus, we slightly moved away from the standard approach to a systematic literature review with only one query for all review questions and divided our search strategy into three queries, each of which corresponded to its own RQ.

The research under RQ1 works with a set of scientific papers that contains a list of QAs specific to ML-enabled system(s). In the literature, they can be represented explicitly as a list (*e.g.*, a study of Habibullah et al. (2023)) or addressed when describing a certain problem or proposing a solution on a system level (*e.g.*, a study of Vojíř and Kliegr (2020)). Preliminary research has shown that in the literature on deep learning systems, neural networks, or artificial intelligence systems, the term "machine learning" may not be explicitly stated in the text of the work. Therefore, we decided to expand the query with the above terms to cover a larger number of papers. The introduction of other ML-related terms (such as "MLOps", "ML Engineering" etc.) could potentially shift focus from architectural scope to a more operational one, while the introduction of other software engineering terms (such as "software quality") could exclude certain papers that did not explicitly mention them. Therefore, we decided not to include those keywords. The resulting query for RQ1 is presented below:

```
("machine learning" OR "deep learning" OR "artificial intelligence" OR "neural network" OR "AI" OR "ML" OR "DL") AND ("system" OR software") AND ("quality attribute*" OR "quality characteristic*" OR "nonfunctional requirement*" OR "nonfunctional requirement*" OR "quality model" OR "quality requirement*")
```

Answering RQ2 identifies architectural tactics that improve certain quality attributes. We used search queries based on the results obtained from RQ1, which included common quality attributes (for example, security), together with their sub-characteristics (respectively, privacy). The difficulty of this task is that relevant tactics are not easily identified, since developers might introduce an architectural tactic without referring to it as such. To address this challenge we also included the terms "design pattern" and "architectural decision" in the query. However, we still consider this challenge as a threat to validity and cannot argue that the list of collected architectural tactics is complete. Search queries for RQ2 were built according to the template presented below with changing parameters of quality attributes together with their sub-characteristics:

The resulting query for RQ1 is presented below:

```
("machine learning" OR "deep learning" OR "artificial intelligence" OR "neural network" OR "AI" OR "ML" OR "DL") AND (system" OR "software") AND ("common quality attribute" OR "subcharacteristic[1]" OR... OR "subcharacteristic[n]") AND ("*architectur* tactic*" OR "design pattern*" OR "*architectur* design decision*" OR "*architectur* decision*")
```

The research under RQ3 implies the study of all possible impacts (predominantly positive, predominantly negative, or ambivalent) of the identified architectural tactics from RQ2 on the common quality attributes identified in RQ1. For RQ3 we wrote 16 queries (equal to the number of identified architectural tactics). We expected that the connections between some ATs and some QAs would not be addressed, however, the papers that brought some insights are of special usefulness for the current research. The structure of the search queries corresponds to the template presented below and includes all of the studied common quality attributes and their sub-characteristics together with a changing parameter of architectural tactic:

```
("machine learning" OR "deep learning" OR "artificial intelligence" OR "neural network" OR "AI" OR "ML" OR "DL") AND ("common quality attribute[1]" OR ... OR "common quality attribute[n]" OR "subcharacteristic[1]" OR... OR "subcharacteristic[m]") AND ("architectural tactic[i]") AND ("trade-off*" OR "tradeoff*" OR "tradeoff*" OR "compromise*")
```

We executed the queries sequentially. The results of data extraction from the sources found with the RQ1-query became the input data for the RQ2-queries, the results of which, similarly, became the input for the RQ3-queries. The full search queries for RQ1, RQ2, and RQ3 as well as the process of data collection are presented in the supplementary artifact[1].

Overall, applying the search procedure with the described queries as well as exclusion and inclusion criteria led to the identification of 206 papers, 37 of which were studied under RQ1, 73 were under RQ2, 96 were under RQ3, and 7 were found for RQ2 but were also found for RQ3 and used to address it.

3.5. Data synthesis

We now discuss the dedicated data synthesis strategies used for each research question as well as our measures taken for ensuring consistency of the data synthesis process.

RQ1. The coding strategy for RQ1 is based on content analysis (Drisko and Maschi, 2016) together with basic frequency analysis and taxonomic analysis (Reed, 2016). First, we employed content analysis to scrutinize the full-text papers to identify possible quality attributes relevant to ML-enabled systems. In the context of our research, content analysis is a manual research method that examines full texts and concepts of scientific papers, allowing us to comprehensively detect relevant quality attributes across the studies. In order to extract a certain characteristic mentioned in a paper as a quality attribute, we introduced two main conditions: "*the characteristic must be explicitly mentioned in the paper*" and "*the characteristic must describe the quality of the overall system*" (not a certain algorithm or component).

In parallel, we detected that the number of identified attributes was going to be quite large, however, some of them were mentioned only in a few papers. This fact introduces a threat to the generalizability of our findings since such attributes can potentially describe the specifics of only one specific system. To avoid this threat, we made a scoping decision based on the hypothesis: *The more often an attribute is mentioned in different independent papers, the more cases it covers, and therefore the more generalizable it is*. To count those mentions we employed a basic frequency analysis. Our basic frequency analysis can be considered as a form of coding, where the code of a quality attribute is defined as the number of papers mentioning it. It is worth noting that all the papers had equal weight when extracting attributes. One quality attribute could be mentioned explicitly either once or several times in the text of the one paper, however, it did not affect the calculated frequency. This algorithm was applied to all papers found, resulting in a ranked list of quality attributes. Based on the resulting counters, a dividing line was drawn between the frequently mentioned and less frequently mentioned quality attributes. The latter were not included in the common quality model.

We noticed that several frequently mentioned attributes were semantically closely related (*e.g.*, reliability and trustworthiness) or by definition can be deemed a superset of several other quality attributes (*e.g.*, maintainability usually covered concerns connected to testability, transparency, and maintainability itself). This observation motivated us to employ taxonomic analysis and group quality attributes by semantic similarity to structure the resulting quality model. First, we formulated high-level definitions that discarded the specifics of individual papers, while retaining the fundamental meanings of attributes. Where it was possible, we directly referred to ISO standards (International Organization for Standardization, 2011a, 2023). In other cases we analyzed extra literature to build proper definitions of found attributes. In the studied papers the definitions of quality attributes usually were not mentioned explicitly. Therefore, we analyzed the selected articles again and checked whether our definitions corresponded to the attribute meanings that were implied in them and whether they were relevant in the context of these papers. When the definitions were formulated in a way that satisfied all the cases, we systemized them. We distinguished quality attributes of two levels based on a principle: *"If one quality attribute covers related concerns with certain other attributes and by definition is broader than them, then such an attribute was considered a ("top-level") common quality attribute, while the other associated attributes were deemed "sub-characteristics"*. We note that during the research under subsequent RQs, both common quality attributes and their sub-characteristics are included in search queries. Therefore, the main goal of the taxonomic analysis was to build a clearer perception of the resulting quality model, which is presented graphically as a two-level diagram.

RQ2. Data synthesis and coding strategies for RQ2 were based exclusively on content analysis. Our goal was to explore all relevant ATs we could find with our search strategy for the scope of common quality attributed as determined in RQ1. Therefore, we did not introduce frequency analysis or taxonomic analysis for RQ2. We thoroughly analyzed full-text papers and followed three conditions for extracting data as ATs: the decision must be explicitly mentioned in a paper, the decision must be architectural in nature (it has an impact on the architectural design principle or can be implemented as a part of the overall system architecture) and the decision must be used to improve some quality attribute(s). Those conditions were introduced with a direct connection to the definition of AT used in this research (see Section 2). If an AT is described as effective in achieving multiple quality attributes, it is associated with all affected attributes. To increase generalizability and eliminate bias, the degree of "significance" of an architectural tactic for a particular attribute was out of scope. For example, if the literature found for RQ2 confirms that the architectural tactic of "containerization" significantly improves both maintainability and portability, then the tactic will be assigned to both attributes, without investigation of which indicator is improved more significantly.

We noticed that some collected tactics only affect the *training system* (*e.g.*, federated learning is usually referred to as a way of organizing model training exclusively), while others can additionally affect other parts of the *deployed system* (*e.g.*, componentization can be the approach to overall system design or be used only to break down the ML pipeline or even the model into components) or be applied to the model when the system is already deployed (*e.g.*, automated bias mitigation usually monitor the outputs of model when it operates with certain inputs). In this context, the *training system* is a system associated with the ML pipeline, which operates with data for model training, testing, and verification; while *deployed system* is a produced ML-enabled system that operates with certain inputs (*e.g* real-time data).

ML-enabled systems may include the training system into the overall architecture to introduce continuous retraining and improvement based on new data (Peldszus et al., 2023). However, in some cases, the training system can be relatively independent. Therefore, we decided to introduce a classification of the identified tactics depending on which system they affect: training or deployed. Our findings were presented in tabular format.

It is important to note that the results obtained to some extent generalize the experience described in the literature, which means if a tactic was described as effective for at least one type of ML-enabled system (for example, an IoT system), it was included in the table. Consequently, we cannot guarantee with full certainty optimal efficiency for other types of machine learning systems, which is also considered in the analysis of threats to validity.

RQ3. For RQ3, we employed content analysis to identify trade-offs that indicate the impact of implementing architectural tactics on quality attributes. A full-text analysis of the papers identified through our search strategy was performed. We reported an impact of a tactic on a quality attribute if at least one source indicated that applying the tactic influenced metrics or other indicators for that attribute. When all sources agreed on the impact's direction, either *predominantly positive* or *negative*, we reported it as such. If sources reported both predominantly positive and negative impacts for the same tactic-quality attribute combination, depending on conditions of the environment or domain, we marked the impact as *ambivalent*. In cases where no evidence of a correlation between an AT and a QA was found, we noted this absence of evidence.

Data extraction consistency. Towards ensuring data extraction consistency, we took three measures.

First, we followed the specific advice from the Kitchenham guidelines for performing SLRs (Kitchenham and Charters, 2007). According to them, it sufficient to conduct *"a test-retest process where the researcher performs a second extraction from a random selection of primary studies"*. This second extraction was conducted by Author 1 on a random sample of *10 papers* for RQ1, *15 papers* for RQ2, and *20 papers* for RQ3. The results of this extraction round were identical to the previous attempt for all RQs.

Second, we continuously discussed the data synthesis and its results in the group of authors. Author 1 strictly followed selected search and data synthesis strategies for RQ1, RQ2, and RQ3 sequentially. Whenever a synthesis of results for a particular RQ was completed, a group discussion with all authors was organized. Author 2 and Author 3 based on their expertise provided feedback on whether the search strategy was executed correctly and whether extracted QAs, ATs, or trade-offs corresponded to selected definitions and conditions for their extraction. At each meeting, the review protocol was presented and updated based on the results of the discussion.

Third, the used literary sources are shared in the publicly available supplementary artifact allowing other researchers to follow our algorithm and analyze selected papers. This also enhances the reproducibility of this research.

3.6. Results verification

All the results should be verified by the experts and practitioners to check their relevance for industrial use. We followed several scenarios of validation depending on the contribution.

Our findings for RQ1 which were compiled in the format of the quality model were verified through:

1. *Expert Validation*. The model was presented at the Swedish Requirement Engineering meeting (SiREN 2023). This event brought together academic and practical experts with a background in the field of requirements engineering and machine learning. An assessment was organized in a focus-group setting with oral feedback. Six experts were surveyed sequentially on three main questions:
    - If the proposed model is *complete*, *i.e.* the identified quality attributes exhaustively characterize the quality of ML-enabled systems.
    - If the proposed model is *general*, *i.e.* the identified quality attributes are applicable to all types of ML-enabled systems, not only to a certain one.
    - If the proposed model is *relevant*, *i.e.* the identified quality attributes respond to current challenges in ML-enabled software quality assurance.
2. *Practitioner Validation*. The model was presented to four ML engineers from Swedish AI software companies. They checked the proposed model against the key quality characteristics used in their enterprise when designing AI-based systems. The validation used the same evaluation parameters as in the case of expert assessment: completeness, generalizability, and relevance.

The findings for RQ2 which were combined in the final list of architectural tactics and associated quality attributes were verified through practitioner validation. The list of ATs was presented to four ML engineers from Swedish AI software companies. They assessed the applicability of architectural tactics to solve problems encountered in the design of AI-based systems within their company, as well as their theoretical validity for improving system qualities.

The findings for RQ3 which were summarized in the resulting table of trade-offs were verified through internal peer-reviewing, where each co-author checked the plausibility of the identified impact (or absence of such) based on their expertise. This review step did not result in any changes to the findings. An additional verification by practitioners and experts is desirable, however, it is overly laborious for the current study due to the large number of impacts identified. In Section 5, we propose and discuss a strategy for such validation in future work.

4. Results

4.1. RQ1: Identification of common quality model

We examined 37 scientific sources to obtain a comprehensive list of quality attributes that characterize various ML-enabled systems. Table 1 provides a list of all quality attributes found and the number of their occurrences in all the sources studied. The list is sorted in descending order of occurrences *(#occ.)* of the quality attribute in the papers.

4.1.1. Studies based on interviews and questionnaires

Several works built models based on the results of interviews, questionnaires, and surveys with experts.

The work of Habibullah et al. (2023) contains the most complete list of quality indicators among all the papers studied. The set of QAs was formed through interviews with practitioners in the field of developing ML-enabled systems. The authors collected 37 quality attributes (system non-functional requirements) relevant to product operation, product revision, and product transition, such as efficiency, usability, portability, etc.

Vogelsang (Vogelsang and Borg, 2019) identified the structure of common requirements for ML-enabled systems: functional and non-functional based on the interview results of several data scientists. The group of non-functional requirements (= quality attributes) included: explainability, freedom from discrimination (= fairness), legality, data quantity, and data quality.

To build sustainable AI architectures Kästner and Kang (2020) indicated six main characteristics of quality assurance based on expert assessment: performance, data quality, testability, safety, security, and fairness.

Ağca et al. (2022) conducted a comprehensive survey on trusted distributed artificial intelligence. The focus of that paper was not on creating a certain quality model, however, the research addresses such quality attributes as performance, robustness, and transparency.

Various quality models have been proposed by other authors: based on an interview study with ML-project stakeholders (Liu et al., 2020; Haindl et al., 2022), industry experts (Ishikawa and Yoshioka, 2019; Serban et al., 2020), and based on the mixture of qualitative and quantitative studies including a survey of practitioners (Wan et al., 2019).

4.1.2. Studies based on expert assessments

There is a group of work presenting the quality characteristics of ML-enabled systems axiomatically, *i.e.* the authors list them as relevant or discuss their relevance based on their own expertise. Since we are examining exclusively scientific "white" literature, we consider the authors as experts and find it reasonable to include such works in the list as well.

Yap (2021) stated that ML systems have unique requirements arising from the interaction with humans such as fairness, privacy, safety, and security (covering the ML component and overall system security). The key quality requirement in that context was trustworthiness.

Ozkaya (2020) pointed out that all the knowledge and experience in designing and reasoning about software systems does not immediately apply to AI-system engineering. The author suggested security, usability, privacy, explainability, data quality, and quantity, testability, and robustness as the critical attributes in the successfully designed structure and behavior of AI-enabled systems.

Zhang et al. (2020) provides a comprehensive survey of techniques for testing machine learning systems. Authors defined quality attributes as testing properties, which included correctness, memory and energy

Table 1
All retrieved quality attributes of ML-enabled systems.

QA	#occ.	QA	#occ.	QA	#occ.	QA	#occ.
Fairness	19	Efficiency	12	Ethics	6	Completeness	2
Safety	19	Usability	11	Data quantity	6	Consistency	2
Security	18	Accuracy	10	Traceability	4	Compatibility	2
Explainability	18	Testability	10	Legal	3	Accountability	1
Privacy	17	Correctness	9	Reusability	3	Justifiability	1
Reliability	16	Func. suitability	8	Interoperability	3	Autonomy	1
Performance	16	Interpretability	8	Reproducibility	2	Modifiability	1
Transparency	14	Trustworthiness	8	Integrity	2	Elasticity	1
Robustness	13	Scalability	8	Repeatability	2	Resilience	1
Data Quality	13	Adaptability	6	Retrainability	2		
Maintainability	12	Portability	6	Modularity	1		

efficiency, robustness, and others.

Truong (2023) suggested applying the author's R3E approach to evaluate the state of end-to-end ML systems. The R3E approach consists of robustness, reliability, resilience, and elasticity.

Kuwajima and Ishikawa (2019), Kuwajima et al. (2020) state that all quality attributes from the standard SQuaRE model could and should be applied to the development of ML-enabled software with additional quality attributes from ethics guidelines for trustworthy AI from the European Commission.

Horkoff (2019) summarized a selection of quality attributes presented previously in the work of Habibullah et al. (2023) and created another quality model consisted of eight general quality attributes: accuracy, performance, fairness, transparency, security, privacy, testability, and reliability.

Various quality models have been proposed by other authors: particularly for AI-chatbots (Chen et al., 2022), ML-based systems for Automotive OEM (Poth et al., 2020), Deep Learning Systems (Challa et al., 2020), IoT systems (Chakraborty et al., 2020), Regression-Based ML-systems (Perera et al., 2022), and other types of AI-based systems (Nakamichi et al., 2020; Lwakatare et al., 2020; Smith and Clifford, 2020; Yokoyama, 2019; Barzamini et al., 2022; Khan et al., 2021; Balasubramaniam et al., 2022).

4.1.3. Studies based on the other methodologies

Some papers stem from methodologies, for example, experience reports from particular companies, design science research of particular solutions, applications of common standards to specific cases, or studies of community trends.

Based on the priorities of a particular company in ML-enabled systems development, Cysneiros and do Prado Leite (2020) identified several key quality attributes: trust (a.k.a. trustworthiness), ethics, and transparency.

Washizaki et al. (2019) collected "good/bad" software engineering design patterns for ML techniques to provide developers with a comprehensive classification of such patterns. Their patterns are implemented to directly affect quality attributes, such as performance, reliability, accuracy, and others.

Ahmad et al. (2023) noticed that industry practices use tools that do not enforce requirements engineering for AI and that there are gaps between research and practices in RE for AI. They conclude that the engineering of AI-systems introduced new specs that did not exist in traditional software, which include data quality, data quantity, accuracy, and explainability.

Felderer and Ramler (2021), Felderer et al. (2019) brought together best practices written by software engineers and data scientists. Key quality attributes according to the studies above were: data quality, system accuracy, correctness, interpretability, etc.

Arseniev et al. (2021) applied fundamental software engineering principles to AI systems. They analyzed how various software teams build software applications with customer-focused AI features and which main problems they meet. The authors claimed that a substantial amount of effort is usually spent on data collection and data preparation. Data quality characteristics also reflect the quality of the AI system. In addition to data quality and quantity, the authors worked with the reliability, scalability, and convenience of accompaniment (in the context of the research equals maintainability).

Other practical-oriented solutions described in the scientific literature were a bug benchmark (Morovati et al., 2023) with key affected attributes of testability, traceability and functional suitability, quality assessment and criteria analysis for AI image recognition software (Tao et al., 2019) with an emphasis on data quality and robustness, compositional approach to creating architectural frameworks for distributed AI systems (Heyn et al., 2023), explaining models in AI (Dodge et al., 2019), ontology-based modeling and analysis of trustworthiness (Amaral et al., 2020), ensuring dataset quality (Picard et al., 2020) and other works that operated with quality attributes (Garbuk, 2018), Boenisch et al. (2021).

4.1.4. Synthesized quality model

A contextual cut-off line between "frequently mentioned" and "infrequently mentioned" attributes was drawn based on the number of their mentions in the scientific literature according to the rule: "If the number of occurrences was greater than 4 then the quality attribute was recognized as frequently mentioned, otherwise, as infrequently mentioned".

The next step was to combine semantically similar frequently mentioned attributes into common quality attributes (CQAs).

Some authors used the term "performance", which implied "system accuracy" or "resource efficiency" depending on the context and to avoid misunderstandings, we divided this term into the above two groups during the data extraction process.

We noticed that the papers that mentioned the quality attribute of "efficiency" used it to describe four different cases: efficiency in terms of running time, efficiency in terms of memory costs, efficiency in terms of energy consumption, or accuracy of system output. The first three cases were exceptionally considered as subcharacteristics of resource efficiency, while the last one was also included as the "system accuracy".

A review of the literature showed that "correctness" and "functional suitability" were often used as contextual synonyms; "usability" was a separate attribute from any other; terms "trustworthiness", "reliability", "safety", "robustness" and "scalability" were of the same nature; "privacy" was presented as a subset of "security", "maintainability" consisted of "system transparency", "testability" and "maintainability" itself; "portability" and "adaptability" were the attributes of the same nature; "explainability" and "interpretability" described by highly-related spectrum of issues, "fairness" and "ethics" were used as synonyms with the rare exceptions of non-standard terminology when "fairness" characterized the "trustworthiness" of model's predictions; "data quantity" was often considered as a special characteristic of the overall "data quality".

The result of the semantic unification of frequently mentioned attributes into common quality attributes and their sub-characteristics is presented in Fig. 1.

Our quality model comprises the following high-level common quality attributes: *Functional suitability* is the degree to which a system

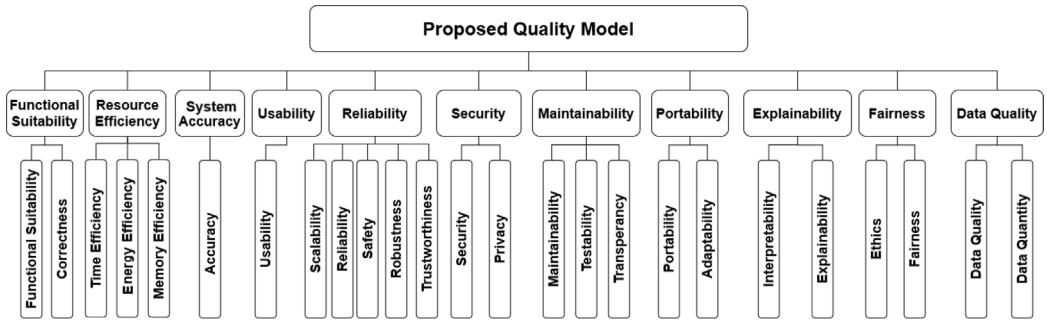

Fig. 1. Proposed Quality Model for ML-enabled Systems.

corresponds to functional requirements. *Resource efficiency* is the degree to which a system fulfills a given functionality within an existing amount of hardware capacities. *System accuracy* is the degree to which a system performs and analyzes the contextual environment and refers to the performance of the entire system in real-world conditions, including model inference and data post-processing. Particularly, system accuracy is different from model accuracy, which specifically measures the predictive performance of the trained machine learning model on a given dataset. *Usability* is the degree to which a system can be employed by end users to achieve specified goals. *Reliability* is the degree to which a system performs specified functions under specified conditions in the fixed domain. *Security* is the degree to which a system protects information and data. *Maintainability* is the degree to which a system can be modified and supported by developers and maintainers to achieve specified goals. *Portability* is the degree of effectiveness with which a system can be transferred from one domain, software, or hardware basis to another. *Explainability* is the degree to which the behavior of a system (primarily, the behavior of ML models) and its output can be explained by humans. *Fairness* is the degree to which a system can detect and prevent an algorithmic bias created by a model. *Data quality* is the degree of integrity and sufficiency of data for model training, testing, and validation, including the reliability of the related data sources.

4.1.5. Verification of the quality model

The developed quality model was presented to six experts at the Swedish Requirements Engineering Meeting (SiREN 2023) in Gothenburg, Sweden organized by Chalmers University of Technology. Four experts out of six participants rated the model as fully complete, general, and relevant. One participant pointed out that the proposed model lacks the "compatibility" attribute (with reference to ISO 25010:2011 International Organization for Standardization, 2011a). According to the used methodology, this attribute was rarely reported in the literature (only 2 times). This fact did not allow us to include it in the final model. This fact was considered as a threat to validity in Section 5. The last expert noted that the selection of terms for quality characteristics is not as important as specific metrics and ways to achieve them, however, they fully supported the proposed model in terms of completeness, generalizability, and relevance.

Further, the model was presented to four ML engineers from Swedish AI software companies. They conducted a theoretical comparison of the quality attributes we found with the system qualities considered by them when designing real AI solutions. Three of the practitioners concluded that the resulting model fully exhausts the quality of all the systems developed in their company and is relevant nowadays. The last expert noted the importance of compliance of the developed solutions with all the quality attributes we identified, as well as with the business goals of stakeholders. According to our findings, meeting business requirements can be expressed in both functional and non-functional requirements. Each NFR can be ranked according to the identified quality attributes, while strict adherence to the functional requirements corresponds to the identified "functional suitability" attribute. After clarifying the context and terminology, the expert agreed that the proposed model was complete, general, and relevant.

4.2. RQ2: Architectural tactics

In Table 2 we present our findings under RQ2 and provide a correspondence in the format of "quality attribute - architectural tactic(s)". We highlight that the table presents only common quality attributes without sub-characteristics, but they were considered in the search strategy, the details of which can be found in the supplementary artifact.[1] Each AT is characterized by its scope as either training system (TS), deployed system (DS) or both. Note that the TS can generally be included as a subsystem in the DS (*e.g.*, in systems that support retraining); in such cases, DS refers to the tactic being applicable to *other* subsystems than the TS.

We note that the "functional suitability" attribute was not included in the summary table for two reasons: this attribute is general in meaning (all ML-enabled systems must follow functional requirements), but not general in the ways of its achievement (for each system there is a unique set of functional requirements), and also because we were unable to find common architectural tactics to satisfy this attribute with the selected methodology.

Below we provide detailed explanations of all the tactics found. To introduce a basic understanding of how each tactic can be implemented, we supplement all the tactics with one example of its implementation from the literature.

4.2.1. Tactics associated with resource efficiency

The ATs to increase resource efficiency were aimed at distributing and reducing resource-intensive processes. The first architectural tactic we found was an approach to distribute the powers for model processes among several computers as known as *Distributed Learning*. Shi et al. (2020) described distributed deep learning as a time and resource-efficient approach to designing the machine learning process. Considering the context in which this tactic is useful, Rao et al. (2011) based on the experiments concluded that distributed learning is effective when there is a need to allocate training loads evenly. The implementation of distributed logic itself (building extra connections) does not sufficiently affect the overall resources consumed. Other papers mentioned that traditional centralized architectures are time-consuming in the domains of biomedical images (Zhao et al., 2023), photonic nanostructures (Noureen et al., 2021) processing and could be replaced with the distribution of loads.

For example, Rao et al. (2011) developed a distributed learning mechanism that enables self-adaptive resource provisioning by treating each virtual machine as a highly autonomous agent that submits resource requests based on its benefit.

The next explored tactic was *Federated Learning (FL)*. In contrast with the centralized approach, FL shifts the computational load to the

Table 2
Architectural tactics to achieve Common Quality Attributes (CQA).

CQA	Tactic	Scope	CQA	Tactic	Scope
Resource Efficiency	- Distributed Learning	TS	Data Quality	- Data Preprocessing	TS/DS
	- Federated Learning	TS		- Data Profiling	TS
	- Automated Data Reduction	TS/DS			
System Accuracy	- Automated Hyperparameter Tuning	TS	Explainability	- Local Interpretable Models	TS/DS
	- Automated Algorithm Selection	DS		- Rule-based Models	TS/DS
Usability	- Human-in-the-Loop	TS/DS	Maintainability	- Componentization	TS/DS
				- Containerization	DS
Security	- Intrusion Detection	TS/DS	Fairness	- Automated Bias Mitigation	DS
	- Automated Data Encryption	TS/DS			
	- Federated Learning	TS			
Reliability	- Automated Data Versioning	TS/DS	Portability	- Containerization	DS
	- Human-in-the-Loop	TS/DS		- Componentization	TS/DS

TS = Training System; DS = Deployed System.

user equipment. While distributed learning involves training models collaboratively across decentralized nodes with mostly shared data, federated learning specifically focuses on training models mostly on local data. Considering application contexts, FL can be profitable when there is a need to lighten the load on the server (Drainakis et al., 2023) during model training.

For example, Brisimi et al. (2018) developed a federated learning framework that can learn predictive models through peer-to-peer collaboration without raw data exchanges solving a binary supervised classification problem to predict hospitalizations based on clients' data.

Finally, we found the tactic of *automated data reduction* relevant for resource efficiency. This tactic can be used in the ML pipeline to reduce training data (Singh and Chaudhari, 2020) or to reduce large amounts of input data in real-time when the system is deployed and operates (ur Rehman et al., 2016). Considering application context, this tactic is useful in systems with massive amounts of data, such as those in the Internet of Things (IoT) (Singh and Chaudhari, 2020) domain. The main perspectives of big data reduction are noise-cleaning and addressing the "curse of dimensionality" caused by millions of variables in big datasets (ur Rehman et al., 2016). In terms of limited resources, this tactic can be the only way to run the system (Hussein et al., 2022).

For example, ur Rehman et al. (2016) collected different methods of automated data reduction in the form of big data compression algorithms, dimension reduction methods, and redundancy elimination.

4.2.2. Tactic associated with usability

With our search strategy we could identify only one AT for increasing the usability of ML-enabled systems, which was the integration of a human as a system component, so-called *"human-in-the-loop" (HitL)*. Petrelli et al. (2012) stated that usability indicators of AI-based systems were sufficiently improved after the integration of so-called "interaction designers" into the processes. They acted as experts to coordinate reciprocal understanding. Winter et al. (2023) and Kröll and Burova-Keßler (2021) viewed close interaction and co-integration between users and the model as mutually beneficial. The user becomes more proficient in using machine learning for their tasks and gets a clearer picture of basic AI principles, while their feedback can serve as a basis for improving the system in terms of usability as well as fairness, explainability, and data quality. According to Heimerl et al. (2012) integration of experts in the training system (when the experts manually evaluate and update training datasets) is as beneficial in terms of usability as their integration in the deployed system (when they evaluate system outputs). Sperrle et al. (2021) proposed a human-centered approach to the evaluation of ML-enabled systems and highlighted the necessity of HitL to improve system usability. Considering the application context, the tactic is especially relevant for accessibility-critical systems (Petrelli et al., 2012) or systems primarily oriented towards the end users (Heimerl et al., 2012).

For example, Gómez-Carmona et al. (2024) noticed that the ML-enabled system in a specific case overlooked human factors (such as human workload or timing) which were critical for end-users. To address this issue, the model was replaced with an adaptive one to consider parameters provided by specific users.

4.2.3. Tactics associated with reliability

To address system reliability or its sub-characteristics we found an AT of *Data Versioning*. Lewis et al. (2021) conducted a complex study on architectural challenges in ML-enabled systems and among other fundamental conclusions, stated that the technique of data versioning can be effective in improving reliability and robustness that serves as a safety net in case of unexpected failures during data processing. This aspect covers mostly the training system. However, in addition to versioning of datasets, it is also important for architects to consciously version related artifacts, such as parameters for model training, data for model evaluation, and evaluation results ("co-versioning") (Van Der Weide et al., 2017). Warnett and Zdun (2022) proposed the versioning of output data when the system is operating to ensure the traceability and integrity of results, allowing for the accurate tracking of changes in outputs over time. Considering the application context, such a tactic is relevant for all systems where multiple versions of a model could be deployed, either over time or even in parallel (Rajendran et al., 2021).

For example, Van Der Weide et al. (2017) used data versioning in the format of saving and storing different versions of end-to-end machine learning pipelines (including datasets for data processing pipelines and model coefficients for model training pipelines) to ensure that multiple versions of a pipeline can run in parallel.

Another tactic to address reliability and safety concerns was previously introduced *Human-in-the-Loop (HitL)*. Rajendran et al. (2021) stated: "The involvement of humans during the training phase can play a crucial role in mitigating some safety issues of autonomous systems", although it can also lead to extra expenses for the vendor. Considering application contexts, integrating expert users is reasonable to validate solutions that need to be available at sufficient capacity (Rajendran et al., 2021).

For example, Rajendran et al. (2021) explored different integrations of experts as components to improve the reliability of deep learning autonomous systems such as "learning from demonstration", "learning from intervention", and "learning from evaluation" to deal with unforeseen circumstances and define safer policies.

4.2.4. Tactics associated with security

The most frequently considered AT for improving security was the integration of *Intrusion Detection*. Sanju (Sanju, 2023) claimed: "The protection of IoT systems from attacks and the assurance of their security posture is ensured by intrusion detection systems". Liu et al. (2022a), Qu et al. (2017), Laqtib et al. (2019), Rashid et al. (2023), El Balbali and Abou El Kalam (2023) listed several fundamental benefits of using intrusion detection as microservices in the architecture of industrial IoT for enhancing security. Intrusion detection is mainly used

for preventing data poisoning (where exclusively the training system is under attack) (Rashid et al., 2023) and adversarial attacks (where minor malicious changes in input data can cause the model to make incorrect predictions) (Qu et al., 2017). In comparison with traditional software, intrusion detection in ML-enabled systems is more flexible and adapted to changes due to possible different outputs produced by the model over time or new behavior of the model caused by retraining. Considering the application context, the tactic is useful for systems operating with large amounts of real-time input data or big datasets consumed by training system (Laqtib et al., 2019), including IoT systems (Sanju, 2023).

For example, Sanju (2023) introduced a hybrid metaheuristics-deep learning approach with an ensemble of recurrent neural networks to detect and prevent intrusions in real-time data processing in an operating IoT system.

Another identified AT was *Automated Data Encryption*. McGraw (2020), Bekri et al. (2024) when analyzing risks for the security of ML-enabled systems, highlighted the important role of encryption of training and testing datasets to protect from several threats primarily. Wu et al. (2022a) found big data encryption efficient for system security, however, some encryption methods may not be optimal also for privacy by default. Kantarcioglu and Shaon (2019) along with several non-architectural decisions, found data encryption as one of the ways to satisfy security requirements in industrial solutions when it comes to ensuring the security of all data flows within a deployed system. In addition to the encryption of datasets and input data, the encryption of the model parameters and weights should be conducted. Considering the application context, the implementation of this tactic is especially relevant for highly sensitive systems that operate continuously (Bekri et al., 2024), including privacy-preserving deep learning systems (Aono et al., 2017).

For example, Aono et al. (2017) proposed additively homomorphic encryption to increase the privacy and security of neural networks by allowing computations to be performed on encrypted data without decrypting it, thus protecting sensitive information throughout the processing stages.

Finally, an effective tactic to architecturally increase privacy and security is an implementation of *Federated Learning (FL)*. In addition to the benefits of this tactic for resource efficiency, it is commonly used to "train a massive amount of data privately due to its decentralized structure" (Kim et al., 2021). Zhou et al. (2022) proved the statement above: "The emerging federated learning (FL) offers a feasible solution for the privacy preservation of users' sensitive data in training AI models". In other words, federated learning allows the benefits of data privacy without the need for data to be shared with a central server (Kaur et al., 2023), Zhang et al. (2021). Considering the application context, this tactic is useful for privacy-critical systems (Kaur et al., 2023; Zhang et al., 2021), including personalized big data systems (Zhou et al., 2022).

For example, Zhou et al. (2022) implemented a federated learning algorithm that ensures that sensitive data is not disclosed during model training together with a user-level personalized differential privacy mechanism.

4.2.5. Tactics associated with maintainability

For both traditional and ML-enabled systems, the architectural tactic of *Containerization* has shown its effectiveness in terms of maintainability and system transparency. Rovnyagin et al. (2020) explicitly claimed the positive effect of containerization along with related tools (such as docker or orchestrator) on system maintainability and operability. Kolltveit and Li (2022) stated: "Models packaged in containers are simply run directly as standalone services" which contributes to the enhancement of maintainability. According to Openja et al. (2022): "Docker (which is a containerization service) allows for convenient deployment of websites, databases, applications' APIs, and machine learning (ML) models with a few lines of code". Finally, containerization allows applied scientists without advanced knowledge to deploy models and access High-Performance Computing (HPC) (González-Abad et al., 2023). Considering the application context, this tactic is useful for systems that require more isolated dependencies and simplified updates (Rovnyagin et al., 2020; Kolltveit and Li, 2022; Openja et al., 2022)

For example, Openja et al. (2022) identified 21 major categories representing the purposes of existing ML projects using Docker, including those specific to ML models, which in turn reduces the complexity of managing ML models.

Another efficient tactic for enhancement of system maintainability was *Componentization*, which obviously contributes to more transparent testing (Braiek and Khomh, 2020). The componentization can be applied to the overall deployed system architecture, or exclusively to the training system, or even exclusively to a model. Singaravel et al. (2018) stated that "Component-Based Machine Learning (CBML) enhances the capabilities of the monolithic ML models" in terms of transparency. Considering the application context, this tactic is relevant for systems that require constant monitoring and updates from the side of developers or maintainers.

For example, Singaravel et al. (2018) transformed the monolithic ML model in the domain of space exploration into a set of connected components which simplified its further maintenance.

4.2.6. Tactics associated with portability

According to the literature review, both tactics associated with maintainability were also profitable for portability.

Componentization is a relevant architectural tactic for increasing portability (Wonsil et al., 2023). Shadab and Salado (2020) reported that the development of AI logic in the format of reusable components could be an adequate solution to increase portability, however, it also may introduce new risks since "currently there is no framework that guides the selection of necessary information to operate in a system different than the one for which the component was originally purposed". Geyer and Singaravel (2018) concluded that components instead of one monolithic model extend reusability and generalization, which in the context of our research directly contributes to the quality of portability. In terms of portability, both componentization of the overall architecture as well as dividing the ML model into components are profitable. Considering the application context, this tactic is useful for cross-platform compatible systems that are partly or fully transferred from one software or hardware base to another (Wonsil et al., 2023) as well as to systems that are transferred from one environment to another (Geyer and Singaravel, 2018).

For example, Geyer and Singaravel (2018) replaced monolithic models with a component-based approach that develops machine learning models not only for the parameterized design of the whole buildings in the construction domain but also for the design of its semi-independent parts on the lower level of abstraction.

The tactic of *Containerization* also improves the portability of ML-enabled systems by encapsulating all dependencies and configurations (Naydenov and Ruseva, 2022). Considering the application context, this tactic is useful for cross-platform compatible systems, including cloud-based services (Joshi et al., 2024).

For example, Naydenov and Ruseva (2022) studied different container technologies used in ML-enabled systems, such as K8s, K3s, Docker, Rancher, and others allowing the systems to run consistently across different platforms, such as local machines or cloud servers.

4.2.7. Tactics associated with explainability

Local Interpretable Models (LIME) are a tool for solving issues of explainability (XAI) and interpretability (Gongane et al., 2024; Minh et al., 2022), by approximating the behavior of a complex underlying model around a specific prediction using a simpler local model that can *explain* the prediction. The intended application context of LIME is a system with a "black-box" model that is inherently non-interpretable,

such as a neural network obtained by deep learning (Maan, 2022). Such complex models cannot be avoided in domains that deal with particularly complex phenomena, such as weather forecasting (Höhlein et al., 2020) and intrusion detection (Gaspar et al., 2024), where LIME is particularly useful. LIME is model-agnostic, which means it can be applied to any ML model without knowing its internals as it only requires access to prediction probabilities. Integrating LIME into a training system can help explain how the model comes to conclusions before deployment and help with the selection of the final model to be used in production. It can be also used in an already deployed system to explain the behavior of the existing model (Saadatfar et al., 2024).

For example, Saadatfar et al. (2024) proposed a LIME algorithm that generates more focused data samples close to the decision boundary and simultaneously close to the original data point in comparison with different LIME implementations, such as BayLIME, SLIME, and LS-LIME.

Another widespread tactic found was the usage of *Rule-based Models* (Burkart and Huber, 2021; Love et al., 2023). While LIME explains individual predictions of complex models by fitting simpler models locally around specific instances, rule-based models directly encode prediction rules that are clear for a human (Moraffah et al., 2020). The intended application context is one where that permits such rules to be formulated, which then inherently leads to explainability and interpretability, and can make rule-based models favorable over more complex ones, especially for non-complicated tasks (Vieira and Digiampietri, 2022). Rule-based models can also be used for rule-based approximation and visualization (Soares et al., 2020). This AT can be also used in both training and deployed systems (Rajapaksha et al., 2020).

For example, Rajapaksha et al. (2020) developed a model-agnostic rule-based approach that obtains k-optimal association rules from a neighborhood of the instance to be explained.

4.2.8. Tactics associated with system accuracy

Automated Hyperparameter Tuning can be especially profitable in increasing model(s) accuracy resulting in the enhancement of the overall system accuracy. It is well-known that the accuracy of machine learning models relies on hyperparameter tuning (Rimal et al., 2024). Daviran et al. (2021) stated: "The predictive accuracy of models can significantly increase when the optimized hyperparameters are predefined and then adjusted to training procedure", which, in this tactic, is an automated process. Considering the application context, this tactic is especially valuable for systems with complex models or large datasets where manual tuning is ineffective, including deep learning systems (Ottoni et al., 2023).

For example, Ottoni et al. (2023) proposed a framework for automated hyperparameter tuning and based on the experimental results proved that this tactic sufficiently improved different accuracy metrics of an image recognition deep learning system.

Another tactic for system accuracy is an *Automated Algorithm Selection*. Kerschke et al. (2019) proposed their implementation of automated algorithm selection from a pre-defined set of algorithms and noted that the choice might be made not only to maximize the accuracy but also based on other contextual priorities. Pise and Kulkarni (2016) listed several key factors that must be considered in a proper algorithm selection tactic. Such a tactic fundamentally improves accuracy in healthcare and medical systems as well (Rashidi et al., 2021; Deeba and Patil, 2021). Considering the application context, this tactic is generally useful for the systems in which high accuracy is particularly important (Kerschke et al., 2019; Pise and Kulkarni, 2016; Alissa et al., 2023), and more specifically in systems operating in constantly changing environments (Alissa et al., 2023).

For example, Alissa et al. (2023) proposed a technique for automated algorithm selection, applicable to certain optimization domains in which implicit sequential information is encapsulated in the data. Specifically, they trained two types of recurrent neural networks to predict a packing heuristic.

4.2.9. Tactic associated with fairness

Automated Bias Mitigation is a common term for a set of algorithms developed to increase the fairness of the deployed ML-enabled system outputs. Lee and Singh (2021) conducted a review of so-called fairness toolkits with the analysis of their relevance in improving system outputs from the ethical perspective. Ferrara et al. (2024) suggested that "building specific methods and development environments, other than automated validation tools, might help developers treat fairness throughout the software lifecycle". Other algorithms for automated bias detection and mitigation were proposed by Agarwal and Agarwal (2023), Castelnovo et al. (2022) and Zhang et al. (2023). Considering the application context, this tactic is primarily useful for the systems operating with personal data and sensitive parameters (Maan, 2022), in particular those that are not inherently interpretable, such as deep learning systems (Maan, 2022).

For example, Maan (2022) proposed a method that evaluates the fairness of deep learning model behavior against sensitive attributes (*i.e.* age, race, gender, sex, and so on) to help mitigate biases without compromising much on accuracy.

4.2.10. Tactics associated with data quality

The tactic of *Automated Data Profiling* is considered an effective tool to increase data quality in the domain of ML-enabled systems (Siddiqi et al., 2023). Data profiling contributes to the training system allowing the identification of missing values and the detection of outliers and anomalies. Considering the application context, this tactic is generally useful for systems that deal with heterogeneous and complex data and, therefore, require comprehensive evaluation to ensure sufficient data quality for training a high-quality model, including domains such as cybersecurity (Canbek et al., 2018), digital twins (Mostafa et al., 2021), and healthcare systems (Logothetis et al., 2022).

For example, Pansara et al. (2024) proposed to employ extra machine learning algorithms to automatically profile and cleanse master data for complex model training operating in the domain of environmental sustainability.

While data profiling implies examining the structure and the content of data to understand its features, *Automated Data Preprocessing* includes data cleaning and data transforming to prepare it for analysis. Gawhade et al. (2022) and Ramkumar et al. (2023) proposed computerized data preprocessing algorithms to primary process input data before it enters the ML model in the deployed system. Santos and Ferreira (2023) suggested another implementation of automated data preprocessing used for the preparation of training datasets in supervised machine learning. In terms of ML-enabled systems, data preprocessing includes data splitting (dividing data into training, testing, and validation datasets). Considering the application context, this tactic is necessary for all types of ML-enabled systems that are going to be trained on unprepared datasets (Santos and Ferreira, 2023) or that operate with raw data on the input (Ramkumar et al., 2023).

For example, Bilal et al. (2022) proposed an automated pipeline for advanced data preprocessing steps of target discretization and sampling which are validated using RandomForest.

4.2.11. Definitions of architectural tactics

Below we present brief contextual descriptions of all architectural tactics found. These definitions were built based on the experience from literature and brought to the common format (relevant for all studied papers despite specifics and context).

AT1: Distributed Learning is an architectural approach to machine learning aimed at parallelizing computing powers among several computers (Rao et al., 2011).

AT2: Automated Data Reduction is an automated process aimed at minimizing the complexity and size of datasets while preserving their essential information (widespread in IoT) (Singh and Chaudhari, 2020).

AT3: Federated Learning is an architectural approach to machine learning aimed at training on local heterogeneous datasets (Drainakis

et al., 2023).

AT4: Human-in-the-Loop (HitL) is an architectural approach where a human (expert) is integrated into the ML-enabled system as a separate component aimed at monitoring and improving the system's behavior (Petrelli et al., 2012).

AT5: Automated Data Versioning is an automated process aimed at the creation, tracking, and management of different versions or iterations of datasets used for model training, testing, and validation (Warnett and Zdun, 2022).

AT6: Intrusion Detection is a tactic for complex systems (primarily, IoT systems) aimed at the detection and classification of network intrusions and anomalies (Sanju, 2023).

AT7: Automated Data Encryption is an automated process aimed at securing sensitive data used for training, inference, or model deployment to protect it from unauthorized access (McGraw, 2020).

AT8: Containerization is an architectural approach aimed at packaging an entire system or model (incl. its dependencies and runtime environment), into a standardized unit called a container (Rovnyagin et al., 2020).

AT9: Componentization is an architectural approach aimed at breaking down a software system into modular components or building blocks that can be independently developed, tested, and deployed (Wonsil et al., 2023).

AT10: Local Interpretable Models (LIME) is an approach to machine learning aimed at explaining black boxes by approximating the behavior of a complex model around a specific prediction using simpler (more interpretable) models (Gongane et al., 2024).

AT11: Rule-based Models is a type of model that relies on explicit rules (*i.e.* if-then) that are designed and specified by humans or domain knowledge to approximate complex model behavior (Love et al., 2023).

AT12: Automated Hyperparameter Tuning (or Hyperparameter Optimization) is a method aimed at searching for the best hyperparameter values for the model based on certain criteria (Ottoni et al., 2023).

AT13: Automated Algorithm Selection (or Algorithm Configuration) is an automated process aimed at searching the most appropriate method(s) for a certain task or in certain circumstances (Kerschke et al., 2019).

AT14: Automated Bias Mitigation is an automated process aimed at identifying and reducing bias in algorithms, models, and datasets by their evaluation through fairness metrics or "sensitive" feature monitoring (Ferrara et al., 2024).

AT15: Automated Data Preprocessing is an automated process aimed at preparing raw data for analysis and model training (Siddiqi et al., 2023).

AT16: Automated Data Profiling is an automated process aimed at analyzing and summarizing the characteristics of a dataset to gain insights into its structure, quality, and distribution (Gawhade et al., 2022).

4.2.12. Verification of the architectural tactics

All authors of this article were conducting constant peer-reviewing of the resulting list of ATs. During weekly meetings, architectural tactics were discussed against identified quality attributes based on the expertise of each co-author. During the validation, issues related to the architectural nature of the identified artifacts and the advisability of classifying them as tactics were discussed. In other words, based on the studied literature we checked if the artifacts affected the overall principle of architectural design (*e.g.*, componentization) or could be integrated as constituent parts into the overall system architecture (*e.g.*, automated bias mitigation module). In this paper, we presented a list of ATs agreed upon by all co-authors of this work.

Further, the list of ATs was shared with four ML engineers from Swedish AI software companies. One practitioner stated that he had experience with all of the suggested tactics to "a greater or lesser extent" except federated learning. He concluded that, based on his experience, all of the tactics presented were relevant to the quality attributes associated with them with an exception for federated learning. Due to insufficient expertise, the interviewee could not confirm or deny this connection. The second practitioner had a similar background and experience with all of the tactics listed except federated learning. He concluded that the list of tactics was accurate and consistent with quality attributes, however, he noted that the list was not complete. He proposed supplementing the list with the tactic of "code versioning" to improve reliability and maintainability. Using our methodology, we were unable to find this tactic relevant in the literature. However, from a practical point of view, we see the importance of this remark. It requires additional assessment and refinement of the search string. The other two experts confirmed their experience in employing all the listed ATs and found all of them relevant in the context of corresponding QAs. They provided several organizational decisions on how to improve resource efficiency and fairness (*e.g.*, evolution of developing culture), however, they struggled to propose any additional ATs to this list.

To enhance the generalizability of verification results, it is preferable to continue validating the list of ATs by experts and practitioners. We anticipate that the list of architectural tactics will expand as we receive feedback from experts. We see great potential in updating the list of tactics and further exploring new entries.

4.3. RQ3: Trade-off analysis

Table 3 represents the summary of our findings obtained during the systematic literature review to answer RQ3.

Below we provide an analysis of the papers which investigate quality trade-offs of the identified architectural tactics.

4.3.1. AT1: Distributed learning

Distributed learning can have positive side-effects on system usability by enabling learning across multiple nodes, thereby enhancing responsiveness and adaptability to diverse user needs (Nassef et al., 2022). However, the impact of distributed learning on privacy (in the current context: on security as well) is controversial. On the one hand, due to their distributed nature such systems are more stable in terms of security since they do not rely only on one server (Cheng et al., 2019) and they can be profitable to preserve privacy due to the essence of decentralized nodes without a necessity to share sensitive information centrally (Zerka et al., 2021). On the other hand, issues with data confidentiality, security breaches, and potential misuse of personal information are connected to increased exposure of sensitive data across those decentralized nodes (Guijarro-Berdiñas et al., 2011; Mandela et al., 2023). The positive impact of distributed learning on portability is proved by its ability to transfer and deploy trained models across different computing environments and devices (Nassef et al., 2022). The negative impact of distributed training on explainability arises from the difficulty of tracking and understanding how individual augmented data from different nodes influence the final results of the model (Tuladhar et al., 2023). Finally, representation across decentralized nodes can lead to biased model outputs and unequal treatment of different demographic classes (Fan et al., 2021).

4.3.2. AT2: Automated data reduction

In addition to the obvious improvement in reducing the load on system resources, automated data reduction tactics also have some limitations. Any interventions in datasets can be risky, especially for complex (low-explainable) models. First of all, such a tactic may decrease system accuracy due to the potential loss of important data during the reduction process (Lane and Brodley, 2019). The risks of incorrect perception of data by the algorithm and classification of useful data as noise or outlier is an obvious risk for data quality and quantity when implementing this tactic (Tomei et al., 2019; Bhuiyan et al., 2019).

Table 3
Trade-off analysis: impact of Architectural Tactics on Quality Attributes.

Quality Attribute	AT1	AT2	AT3	AT4	AT5	AT6	AT7	AT8	AT9	AT10	AT11	AT12	AT13	AT14	AT15	AT16
Resource Efficiency	+	+	+	0	0	−	−	−	0	+	0	−	−	−	a	0
Usability	+	0	0	+	0	0	0	0	0	0	0	0	0	0	+	+
Reliability	0	0	a	+	+	+	+	+	+	−	0	+	+	a	−	+
Security	a	0	a	a	0	+	+	−	0	0	0	+	0	0	+	+
Maintainability	0	0	0	+	+	0	0	+	+	+	0	0	+	0	+	a
Portability	+	0	0	0	0	0	0	+	+	0	0	+	0	+	0	0
Explainability	−	0	0	+	0	−	0	0	+	+	+	0	−	0	+	0
System Accuracy	0	−	−	0	0	0	−	0	+	−	−	+	+	−	a	+
Fairness	−	0	−	+	0	0	0	0	0	+	0	+	0	+	a	0
Data Quality	0	−	−	+	0	+	+	+	0	0	0	0	0	a	+	+

+ = predominantly positive impact, − = predominantly negative impact, a = ambivalent impact, 0 = insufficient evidence either way.

4.3.3. AT3: Federated learning

In RQ2 we identified that federated learning is often used for increasing security and privacy particularly, however, some papers found for RQ3 by Shen et al. (2022), Jeong and Chung (2022), Jagarlamudi et al. (2023) and Shin et al. (2023) point to the practical insecurity of existing implementations of federated learning and noted severe vulnerabilities associated with data leakage or inference attacks during the decentralized model training across multiple devices: "existing federated learning protocol designs have been shown to be vulnerable to adversaries within or outside of the system, compromising data privacy" (Lyu et al., 2022). Therefore, based on the development level of federated learning at the time of writing the current paper, its impact on security and privacy is recognized as controversial. The reliability of federated learning systems can be considered ambivalent. On the one hand, "federated learning resulted in a reliable strategy for model development" (Kirienko et al., 2021) due to its capability to incorporate diverse and decentralized data sources. On the other hand, potential communication bottlenecks and data heterogeneity across devices lead to severe challenges in terms of robustness (Lyu et al., 2022; Sattler et al., 2020; Lycklama et al., 2023). Some risks of federated learning are also connected to system accuracy due to the aggregation of diverse and potentially noisy local data from distributed devices and fairness due to insufficient diversity of data collected (Gu et al., 2022). Such challenges also affect overall system data quality.

4.3.4. AT4: Human-in-the-Loop (HitL)

The effect of HitL on security and privacy is controversial. On the one hand, integration of human intelligence as a system component can bring the benefit in guiding the XAI-enabled system and generate refined solutions in terms of vulnerability detection (Nguyen and Choo, 2021; Jones et al., 2018), and on the other hand, "the involvement of humans results in an external and unpredictable element that increases security concerns" (Jena et al., 2022). Human-in-the-Loop is a unique element that plays a crucial role in the human-centered system qualities such as maintainability by intelligently tracking changes and intermediate results over time (Xin et al., 2018), explainability by leveraging bidirectional symbiotic sensing feedback (Kang et al., 2021; Rodríguez et al., 2024; Zhang et al., 2022) and fairness by identifying sensitive data and parameters (Liu, 2022; Kalananthan et al., 2023; Ghai and Mueller, 2022). Finally, data quality can be significantly improved based on the feedback constantly provided by analysts and engineers (Priestley et al., 2023).

4.3.5. AT5: Automated data versioning

Automated data versioning can enhance the maintainability of ML-enabled systems by ensuring reproducibility and traceability of model training and inference, which simplify debugging and model updates (Jakubik et al., 2024; Yousefi et al., 2023).

4.3.6. AT6: Intrusion detection

Intrusion detection in IoT systems can negatively affect resource efficiency due to the high computational and memory requirements of deep neural networks (Tsimenidis et al., 2022; Devendiran and Turukmane, 2024). While "an intrusion detection system is a promising automotive security enhancement", it also improves anomaly detection capabilities, thereby improving overall system robustness by reducing the risk of non-security-related failures and errors (Lampe and Meng, 2023). The inherent complexity of deep neural networks for intrusion detection negatively affects the explainability of the overall often low-explainable IoT systems (Pawlicki et al., 2023; Shand et al., 2023). Finally, identifying and removing unnecessary data from the datasets significantly contribute to the data quality on a system level (El Balbali and Abou El Kalam, 2023; Naydenov and Ruseva, 2022).

4.3.7. AT7: Automated data encryption

Data encryption in ML-enabled systems can negatively impact resource efficiency by increasing computational overhead and latency due to the additional processing requirements for encryption and decryption (Aljawarneh et al., 2018; Weng, 2023). The positive side-effect of data encryption in terms of reliability appears due to ensuring data integrity and reducing the risk of data corruption (Cantoro et al., 2020). Wang et al. (2021) noted a slight decrease in the system accuracy of the encrypted model in comparison with non-encrypted solution. Any data protection tactic also makes a significant contribution to overall data quality (Gupta and Lakhwani, 2022).

4.3.8. AT8: Containerization

The main challenge of containerization in terms of resource efficiency arises from the potential resource overhead of container orchestration and virtualization (Rovnyagin et al., 2019; Openja et al., 2022). However, such a tactic has a positive side effect on system reliability by providing a consistent and isolated runtime environment (Rovnyagin et al., 2019). Figueroa et al. (2023) claimed: "Combined with IoT, containerization allows efficient allocation, fast execution, and deployment of hardware resources". According to Joraviya et al. (2024): "Containerization has introduced new security challenges including cloud data breaches in ML-enabled systems", which is also accompanied by increased attack surface and potential misconfigurations including increasing risks of data breaches, model theft, and adversarial attacks due to shared resources, image vulnerabilities, and insufficient isolation, making strict access control and monitoring essential. Finally, it has a positive effect on data quality due to its ability to facilitate consistent data handling and processing environments (Arisdakessian et al., 2023).

4.3.9. AT9: Componentization

With our search strategy we could not find any evidence that componentization has any crucial impact on resource efficiency. However, we found a positive impact of this tactic on the reliability of IoT systems (Siddiqui et al., 2023) by enabling the implementation of

safety-critical components with clear interfaces and well-defined behaviors. At the same time, this tactic is reasonable to isolate specific model components to improve explainability and interpretability (Sarjoughian et al., 2023). Finally, based on the experimental results Heisele et al. (2011) concluded that "the component system clearly outperformed global systems on all tests in terms of accuracy".

4.3.10. AT10: Local interpretable models (LIME)

The work of Kumarakulasinghe et al. (2020) significantly contributed to the analysis of trade-offs when applying local interpretable models. According to this study, LIME provides interpretable and simple local explanations without a need for resource-intensive global model explanations, which in some cases is really profitable for resource efficiency. This tactic of simplification also contributes to improvements in maintainability. However, it goes in contrast with reliability, where LIME brings the risks of potentially incorrect local explanations that do not accurately reflect the overall behavior. Mori and Uchihira (2019) also found this fact a reason for misleading interpretations and decisions (which is considered a negative impact on system accuracy). At the same time, LIME can be used to assess a classifier's fairness and to determine the sensitive features to remove (Bhargava et al., 2020).

4.3.11. AT11: Rule-based models

The only impact of rule-based models found with our search strategy was on system accuracy, which can suffer from limited adaptability to complex and dynamic data patterns when scenarios lie outside the predefined rules (Burkart et al., 2019; Soui et al., 2019; Rey et al., 2017).

4.3.12. AT12: Automated hyperparameter tuning

When automated hyperparameter tuning (HPT) is aimed at increasing system accuracy, the trade-off with performance efficiency occurs most often (Liao et al., 2022; Romsaiyud et al., 2019). Liu et al. (2022b) claimed: "The current resource provisioning approaches for HPT are unable to adjust resources adaptively according to the upward trends of HPT accuracy at runtime. On the other hand, dynamic resource provisioning approaches based on checkpointing are inefficient for HPT, because of the high overhead of context switching and job restarting". HPT can enhance reliability by optimizing model generalization, reducing the risk of overfitting (Jain et al., 2023; Kunang et al., 2021). The positive impact of HPT is also noted in terms of security (Wu et al., 2022b; Batchu and Seetha, 2021) by improving resistance against adversarial attacks. Feroz et al. (2024) claimed that HPT can improve not only system accuracy and system reliability but also the adaptability and portability of the system in different real-life scenarios. Finally, HPT can be used in the form of optimizing model parameters to reduce bias and consider different demographic groups or sensitive attributes (Perrone et al., 2021; Gao et al., 2022).

4.3.13. AT13: Automated algorithm selection

Automated algorithm selection like HPT often brings a trade-off between resource efficiency and system accuracy (Dagan et al., 2024; Bossek et al., 2020). The main possible benefit of the automated algorithm selection component lies in the recommendation of a promising learning algorithm based on meta features computed from a given dataset (Shahoud et al., 2021). Such analysis can be too complicated and time-consuming for human data scientists and delegation of those responsibilities to ML is considered a valuable contribution to system maintainability. Also, it in some sense minimizes human factors in algorithm selection, which has a positive effect on reliability as well. However, existing automated algorithm selection methods for increasing accuracy rarely consider explainability as a factor for selection, which leads to the complexity of automatically chosen algorithms (Trajanov et al., 2022).

4.3.14. AT14: Automated bias mitigation

Hutiri et al. (2023) found the risks of computational overhead due to the complexity of bias detection and correction algorithms in the context of IoT systems. The impact of this tactic on reliability is ambivalent. On the one hand, such methods improve system stability when working with diverse datasets, however, they can also lead to potential unintended model changes with risks of system failures (Hort et al., 2023; Ghani et al., 2023). Increased adaptability of the ML-enabled system also influences the common attribute of portability (Jain and Kumar, 2023). The potential distortion or removal of relevant patterns in the data introduced by this tactic harms system accuracy (Hutiri et al., 2023; Chen et al., 2023). This fact also affects the attribute of data quality (Miceli et al., 2022).

4.3.15. AT15: Automated data preprocessing

This tactic has a controversial impact on resource efficiency. According to Ramírez-Gallego et al. (2017): on the one hand, the introduction of automated data preprocessing contributes to a faster and more precise learning process which can potentially save resources, on the other hand, when it comes to big data systems such tactic can lead to resource overload due to the large volumes of data being processed. Rendleman et al. (2019) proposed a method to increase the usability of a certain module when data preprocessing is conducted according to the priorities of end users. Automated preprocessing offers certain benefits in terms of model training, such as lowering the manual effort required for data preparation and enhancing maintainability by structuring and formatting data (Shivashankar and Martini, 2022), while it can also protect models against malicious inputs and data poisoning (Hikal and Elgayar, 2020; Bouke and Abdullah, 2023). It also improves explainability by ensuring consistent and standardized data transformation, which makes model behavior and decision insights clearer to humans (Zelaya, 2019; Basha and Kuppusamy, 2022). The impact of this tactic on system accuracy is ambivalent since it can be either improved by standardizing input data and noise cleaning or harmed by potentially introduced biases or distortions by the algorithms (Sun et al., 2022; Obaid et al., 2019). The complexity of automated data preprocessing in the context of human-centered learning can also have different implications for fairness due to the same reasons (Sun et al., 2022).

4.3.16. AT16: Automated data profiling

With our search strategy we were unable to find any evidence that automated data profiling has any crucial impact on resource efficiency as we expected. However, the personalization of certain data (which is a subset of data profiling) can significantly increase usability according to the needs of certain users (Sajid et al., 2019; Oppold and Herschel, 2020). Data quality improvements provided by automated data profiling modules play a crucial role in overall system reliability (Ding and Mit, 2023). In the context of IoT, data profiling can detect data vulnerabilities and privacy risks as an improvement of system security and upgrade feature understanding as an improvement of system accuracy (Seo et al., 2019). Finally, the impact of data profiling on maintainability is controversial since it can reduce manual effort on data management, but brings the risk of over-reliance on automated processes, which can be "black boxes" if they are executed by complex ML-algorithms (Epperson et al., 2023; Tverdal et al., 2023).

4.3.17. Verification of the trade-off matrix

The resulting table of quality trade-offs was constantly being peer-reviewed by co-authors of this paper based on their independent expertise. This paper presents a version of the table agreed upon by all authors of the article.

All identified trade-offs are supported with literature references, however, more expert validation is desirable. Due to the large number of identified impacts, it would be complex and lead to significant effort. We present a strategy for such assessment in Section 5.

5. Discussion

5.1. Observations regarding quality attributes

This study proposed a common quality model for ML-enabled systems. This model took a broader look at ML-enabled specific nature and suggested considering attributes related to "data quality" and ML-unique "explainability", "system accuracy" and "fairness" along with a standard set of attributes.

The attribute of system accuracy is a complex indicator that goes beyond model accuracy itself. In the context of quality, it is used to understand whether the system can operate effectively in the existing context with the existing accuracy (incl. metrics of precision, recall, F-score, etc.).

Thirteen papers referred to data quality as an attribute of the overall system quality. This attribute characterizes the quality of data sets used for model training and testing, as well as the ways this data was obtained and the sources from which this data was collected from at the system level. Working with unreliable or incomplete data causes a high risk of system failure due to its incorrect or insufficient perception of contextual reality as well as legal issues.

Quality attributes rarely addressed in the reviewed literature may still deserve further study. For any attribute named at least one time in RQ1, we can assume that it is relevant for some specific ML-enabled system(s). For example, the retrainability attribute can be very important for systems operating in especially dynamically updated environments, and the autonomy attribute could be relevant for systems that, for a number of reasons, need to be isolated from all external influences.

Remarkably, the standard quality attribute of compatibility is emphasized in the literature as relevant to the quality of ML-enabled systems only two times. For this reason, this quality attribute was not included in the proposed model. According to studied papers, we noticed that the considered works typically sought to study characteristics connected to external entities: resource efficiency as a result of interaction with available resources, usability as the result of interaction with end-users, maintainability - with developers and maintainers, system accuracy - with context, fairness - with society, portability - with new environments, etc. However, the interaction of ML-enabled systems with other software systems (which is the basis for compatibility) remains poorly described in the scientific literature and likely requires additional attention.

5.2. Comparison to ISO standards

In 2023, the International Organization for Standardization (ISO) issued a new standard ISO 25059:2023 (International Organization for Standardization, 2023). This standard offers a quality model for AI systems, which can be seen as a possible alternative answer to RQ1. A comparison of our quality model with ISO 25059:2023, as well as with the traditional SQuaRE quality model (ISO 25010:2011 (International Organization for Standardization, 2011a)), is presented in Table 4. The table maps all high-level attributes from three quality models grouped by semantic similarity. We now compare our obtained quality model to those from two relevant ISO standards. Importantly, our goal is not to present a new, competing standard, but to reflect the results of our literature review in comparison to existing quality models, in particular, those from the standards.

Factually, neither of the previous standards considers system accuracy and data quality on the system level, whereas we found in RQ1 that the literature frequently mentions them as system-level concerns and in RQ2 identified appropriate architectural tactics to address them. Moreover, ISO 25059:2023 considers the ethical perspective (related to fairness in the terminology of our research) as a high-level quality attribute and combines transparency with explainability. In our model, we decided to separate explainability and transparency. While system transparency refers to the openness and accessibility of a system's internal processes (which is mostly important for developers and maintainers), system explainability focuses on how clearly the system's decisions and processes can be understood and interpreted by humans (which is more important for users, operators, and experts). The release of ISO 25059:2023 confirmed the relevance of RQ1 and highlighted the need to consider ML-specifics in assessing the quality of software systems.

5.3. Observations regarding trade-offs

The analysis of trade-offs proposes a comprehensive mapping of different impacts after the implementation of certain ATs reported by different researchers. The most frequently reported trade-offs appeared between system accuracy and resource efficiency, system accuracy and explainability, fairness, and resource efficiency. Also, notable positive "side-effects" include the facts that: architectural enhancement of security can often increase data quality and reliability, enhancing system accuracy positively affects reliability, and tactics to improve data quality can have a positive impact on usability and security.

We met a remarkable situation with AT3 of Federated Learning. While investigating RQ2, we found its wide application to increase resource efficiency and security. However, during the work on RQ3, we identified several significant vulnerabilities in existing implementations of federated learning, which motivated us to characterize its impact on security as controversial.

Context analysis is critical when applying architectural tactics of distributed learning in terms of security, automated data preprocessing in terms of accuracy and fairness, and automated bias mitigation in terms of reliability and data quality.

5.4. Threats to validity

The main threat to external validity is that the generalizability of our architectural findings can be limited as we restrict our analysis to available literature, specifically, scientific papers. While the analyzed literature stems from diverse domains and methodologies, including those that analyze practical experiences from companies, we cannot claim generalizability to all possible systems and sub-domains. A planned mitigation is to conduct a gray literature study investigating non-scientific literature such as blog entries and repository documentation. Another threat to external validity is caused by the complexity of the search process for RQ2. We searched for architectural tactics with corresponding keywords, however, it is possible that some architectural tactics were not referred to as such in the literature. To mitigate this we also included related terms (*e.g.*, "design decisions") in the search query. However, we still cannot state that the list of found ATs is complete and to mitigate this threat, we plan to conduct some more interview studies with practitioners and experts. Finally, the mapping of quality trade-offs for RQ3 was complex and should take sufficient resources for its validation from the side of practitioners. The possible mitigation is to follow the proposed verification strategy described further.

The main threat to internal validity associated with our approach to consider all quality attributes of equal importance as well as architectural tactics of equal value. Therefore, we do not weigh the magnitude of the trade-offs between quality attributes and the scale of the consequences of AT implementation. It can be mitigated by in-depth research of each AT with the study of the specific contexts, leaving generalizability behind.

The main threat to construct validity is our focus on commonly reported quality attributes in the literature, which we implicitly assume to be correlated with their generalizability and the need to include these attributes in a common quality model for the domain of ML-enabled software. Clearly, some of the less commonly reported attributes might still be important in particular domains and use cases. To mitigate this threat it is possible to run a separate study of such attributes.

Table 4
Comparison of proposed quality model and ISO standards.

Proposed Quality Model	ISO 25059: AI systems	ISO 25010: general software
Functional Suitability	Functional Correctness	Functional Suitability
Resource Efficiency	–	Performance Efficiency
Usability	User Controllability	Usability
Reliability	Robustness	Reliability
Security	–	Security
Maintainability	Intervenability	Maintainability
Portability	Functional Adaptability	Portability
Explainability	Transparency	–
System Accuracy	–	–
Fairness	Ethical	–
Data Quality	–	–
–	–	Compatibility

5.5. Implications for practitioners

Our study has several implications for practitioners and researchers. Considering practitioners, our findings can be used in a checklist-like manner during the system requirements and design stages. During requirements elicitation, our quality model can guide practitioners in identifying relevant non-functional requirements for the overall system that then need to be addressed by the system architecture. The list of architectural tactics provides them with insights into how to achieve those quality attributes architecturally. We especially highlight our description of context and examples we provide for each tactic, which allows practitioners to match the tactics to their particular context at hand. Finally, the table of trade-offs raises awareness of possible unintended consequences of decisions made. These insights can be especially valuable for start-ups and SMEs with limited resources for hiring ML engineering experts.

Our findings are also informative for the emerging area of *MLOps*, which focuses on the integration of DevOps principles and practices into the development and maintenance of machine learning systems (Alla et al., 2021). While a dedicated literature study of quality attributes and tactics for MLOps is outside our scope, we identify the following potential applications of our findings in an MLOps context. First, the identified quality attributes can help align the engineering process with business goals and operational needs and contribute to more sustainable software development. For example, the consideration of maintainability at the design stage can guide the optimization of planned resources for further maintenance, support, and updates of the deployed system (Shivashankar and Martini, 2022). Second, treating training and deployed systems as aspects of a single complex software architecture along with appropriate tactics during the design phase, can improve operations associated with system retraining (Peldszus et al., 2023). Finally, recognizing trade-offs can help allocate resources and prioritize tasks within the common workflow (Barney et al., 2012). For example, if a team uses containerization to improve maintainability, our findings emphasize potential drawbacks for security, which need to be addressed specifically within the roles and responsibilities of the DevOps setting, for example, by actively planning and estimating the effort arising for a security team within the company (Mohan and Othmane, 2016).

5.6. Implications for researchers

We suggest several directions for future work within the existing architectural perspective. First, our list of trade-offs can be informative for researchers to develop new architectural tactics. Given that our numbers of identified tactics per quality attribute are between one and three, there is a potential for new tactics that complement the existing ones to find a different "sweet spot" within the trade-offs, possibly addressing specific domains and application contexts. Second, more generally, while our study was broadly focused on ML-based systems, it would be worthwhile to conduct additional studies that explore the relevance and applicability of our findings in different domains (*e.g.*, automotive vehicles, healthcare systems, etc.). Third, our study of trade-offs is based on literature sources; yet, the results could benefit from follow-up research that verifies and refines the resulting table of trade-offs based on insights in industrial settings.

While we conducted a first evaluation of our results with experts, there is room for further evaluating our findings in complementary ways, focusing on their real-world applicability in specific contexts. As part of future work, we propose to conduct such evaluations with empirical methods, such as case studies, controlled experiments, and surveys. As a promising direction, we highlight *action research* (Staron, 2020), which is dedicated to deploying solutions in a specific real-world context—for example, in our case, specific quality attributes and tactics in an industrial MLOps environment with multiple teams. In such an environment, we aim to investigate if the introduction of a quality model helps teams to allocate their priorities more efficiently, if the identified architectural tactics can be applied to systems of different natures and sizes and be generalizable to new contexts, and how identified trade-offs affect the decisions made by MLOps teams. Such an evaluation may shift the focus from our current architectural perspective to a more operational one, which can be especially relevant in the context of MLOps.

6. Conclusion

This work contributes to the methodology of building software architecture for ML-enabled systems from the perspective of quality, offering ways to define it and achieve it through common architectural tactics with a consideration of possible side effects. Our focus is on theory-building, as we systematically identified and synthesized information from 206 research papers.

There are several worthwhile directions for future work. First, while our contributions are informed and validated by empirical insights from published literature, additional validation, for example, through expert feedback, is possible. Second, the selection of literature for analysis can be expanded by including gray literature. As a result, the quality model and proposed architectural tactics can be refined. Third, one can conduct complementary forms of evaluation highlighting the applicability of our findings in specific real-world contexts, through action research and other empirical methods. The emerging area of MLOps provides a particularly promising avenue for such evaluations. Finally, we suggest follow-up research to further investigate the role of quality aspects mentioned infrequently in the literature (*e.g.*, portability) and to systematically study the impact of the identified tactics on all quality aspects.

CRediT authorship contribution statement

Vladislav Indykov: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Daniel Strüber:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Funding acquisition, Conceptualization. **Rebekka Wohlrab:** Writing – review & editing, Validation, Supervision, Data curation, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Daniel Strüber reports a relationship with Radboud University that includes: employment. Rebekka Wohlrab reports a relationship with Carnegie Mellon University that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation, Sweden, and VR. We are grateful to the experts presented at the Swedish Requirements Engineering Meeting (SiREN) 2023 and ML engineers from the Swedish AI software company for providing expertise on our findings.

Data availability

The supplementary data are now available under the following DOI: https://doi.org/10.6084/m9.figshare.25673643.v1.

References

Agarwal, A., Agarwal, H., 2023. A seven-layer model with checklists for standardising fairness assessment throughout the AI lifecycle. AI Ethics 1–16.
Ağca, M.A., Faye, S., Khadraoui, D., 2022. A survey on trusted distributed artificial intelligence. ECSA.
Ahmad, K., Abdelrazek, M., Arora, C., Bano, M., Grundy, J., 2023. Requirements practices and gaps when engineering human-centered artificial intelligence systems. ASOC.
Alissa, M., Sim, K., Hart, E., 2023. A feature-free approach to automated algorithm selection. In: GECCO. Wiley, pp. 9–10.
Aljawarneh, S., Yassein, M.B., Talafha, W.A., 2018. A multithreaded programming approach for multimedia big data: encryption system. MTA 77, 10997–11016.
Alla, S., Adari, S.K., Alla, S., Adari, S.K., 2021. What is MLOps? pp. 79–124, Beginning MLOps with MLFlow: Deploy Models in AWS SageMaker, Google Cloud, and Microsoft Azur.
Amaral, G., Guizzardi, R., Guizzardi, G., Mylopoulos, J., 2020. Ontology-based modeling and analysis of trustworthiness requirements: Preliminary results. In: ER. pp. 342–352.
Aono, Y., Hayashi, T., Wang, L., Moriai, S., et al., 2017. Privacy-preserving deep learning via additively homomorphic encryption. TIFS 13 (5), 1333–1345.
Arisdakessian, S., Wahab, O.A., Mourad, A., Otrok, H., 2023. Towards instant clustering approach for federated learning client selection. In: ICNC. IEEE, pp. 409–413.
Arseniev, D.G., Baskakov, D.E., Kasurinen, J., Shkodyrev, V.P., Mergasov, A., 2021. Software engineering principles apply to artificial intelligence systems. In: CPS&C. pp. 151–158.
Balasubramaniam, N., Kauppinen, M., Hiekkanen, K., Kujala, S., 2022. Transparency and explainability of AI systems: ethical guidelines in practice. In: REFSQ. pp. 3–18.
Barney, S., Petersen, K., Svahnberg, M., Aurum, A., Barney, H., 2012. Software quality trade-offs: A systematic map. Inf. Softw. Technol. 54 (7), 651–662.
Barzamini, H., Shahzad, M., Alhoori, H., Rahimi, M., 2022. A multi-level semantic web for hard-to-specify domain concept, pedestrian, in ML-based software. RE.
Basha, M.M., Kuppusamy, P., 2022. F-DDPT: An efficient fuzzy-based automated preprocessing technique to support explainability. In: ICCDN. Springer, pp. 283–296.
Bass, L., Clements, P., Kazman, R., 2003. Software architecture in practice. Addison-Wesley.
Batchu, R.K., Seetha, H., 2021. A generalized machine learning model for DDoS attacks detection using hybrid feature selection and hyperparameter tuning. Comput. Netw. 200, 108498.
Bekri, W., Jmal, R., Fourati, L.C., 2024. Secure and trustworthiness IoT systems: investigations and literature review. TS 1–36.
Bhargava, V., Couceiro, M., Napoli, A., 2020. LimeOut: an ensemble approach to improve process fairness. In: ECML PKDD. Springer, pp. 475–491.
Bhat, M., Shumaiev, K., Hohenstein, U., Biesdorf, A., Matthes, F., 2020. The evolution of architectural decision making as a key focus area of software architecture research: A semi-systematic literature study. In: ICSA. pp. 69–80.
Bhuiyan, M.Z.A., Wang, T., Zaman, A., Wang, G., 2019. Data reduction through decision-making based on event-sensitivity in IoT-enabled event monitoring. In: HPCC. IEEE, pp. 2039–2044.
Bilal, M., Ali, G., Iqbal, M.W., Anwar, M., Malik, M.S.A., Kadir, R.A., 2022. Auto-prep: efficient and automated data preprocessing pipeline. IEEE Access 10, 107764–107784.
Boenisch, F., Battis, V., Buchmann, N., Poikela, M., 2021. "I never thought about securing my machine learning systems": A study of security and privacy awareness of machine learning practitioners. In: MuC. pp. 520–546.
Bossek, J., Kerschke, P., Trautmann, H., 2020. A multi-objective perspective on performance assessment and automated selection of single-objective optimization algorithms. ASC 88, 105901.
Bouke, M.A., Abdullah, A., 2023. An empirical study of pattern leakage impact during data preprocessing on machine learning-based intrusion detection models reliability. Expert Syst. Appl. 230, 120715.
Braiek, H.B., Khomh, F., 2020. On testing machine learning programs. JSS 164, 110542.
Brisimi, T.S., Chen, R., Mela, T., Olshevsky, A., Paschalidis, I.C., Shi, W., 2018. Federated learning of predictive models from federated electronic health records. Int. J. Med. Informatics 112, 59–67.
Burkart, N., Huber, M.F., 2021. A survey on the explainability of supervised machine learning. JAIR 70, 245–317.
Burkart, N., Huber, M., Faller, P., 2019. Forcing interpretability for deep neural networks through rule-based regularization. In: ICMLA. IEEE, pp. 700–705.
Canbek, G., Sagiroglu, S., Temizel, T.T., 2018. New techniques in profiling big datasets for machine learning with a concise review of android mobile malware datasets. In: IBIGDELFT. pp. 117–121.
Cantoro, R., Deligiannis, N.I., Reorda, M.S., Traiola, M., Valea, E., 2020. Evaluating data encryption effects on the resilience of an artificial neural network. In: DFT. IEEE, pp. 1–4.
Castelnovo, A., Crupi, R., Greco, G., Regoli, D., Penco, I.G., Cosentini, A.C., 2022. A clarification of the nuances in the fairness metrics landscape. Sci. Rep. 12 (1), 4209.
Chakraborty, A., Bagavathi, R., Tomer, U., 2020. A comprehensive decomposition towards the facets of quality in IoT. In: ICOSEC. pp. 759–764.
Challa, H., Niu, N., Johnson, R., 2020. Faulty requirements made valuable: On the role of data quality in deep learning. In: AIRE. pp. 61–69.
Chen, Q., Gong, Y., Lu, Y., Tang, J., 2022. Classifying and measuring the service quality of AI chatbot in frontline service. J. Bus. Res. (ISSN: 0148-2963).
Chen, Z., Zhang, J.M., Sarro, F., Harman, M., 2023. A comprehensive empirical study of bias mitigation methods for machine learning classifiers. TSEM.
Cheng, H.-P., Yu, P., Hu, H., Zawad, S., Yan, F., Li, S., Li, H., Chen, Y., 2019. Towards decentralized deep learning with differential privacy. In: ICCC. Springer, pp. 130–145.
Cysneiros, L.M., do Prado Leite, J.C.S., 2020. Non-functional requirements orienting the development of socially responsible software. In: CAiSE. pp. 335–342.
Dagan, I., Vainshtein, R., Katz, G., Rokach, L., 2024. Automated algorithm selection using meta-learning and pre-trained deep convolution neural networks. Inf. Fusion 105, 102210.
Daviran, M., Maghsoudi, A., Ghezelbash, R., Pradhan, B., 2021. A new strategy for spatial predictive mapping of mineral prospectivity: Automated hyperparameter tuning of random forest approach. Comput. Geosci. 148, 104688.
Deeba, F., Patil, S.R., 2021. Utilization of machine learning algorithms for prediction of diseases. In: I-PACT. IEEE, pp. 1–7.
Devendiran, R., Turukmane, A.V., 2024. Dugat-LSTM: Deep learning based network intrusion detection system using chaotic optimization strategy. Expert Syst. Appl. 245, 123027.
Ding, N.B., Mit, E., 2023. A framework of data quality assurance using machine learning. In: CITA. IEEE, pp. 88–93.
Dodge, J., Liao, Q.V., Zhang, Y., Bellamy, R.K., Dugan, C., 2019. Explaining models: an empirical study of how explanations impact fairness judgment. In: IUI. pp. 275–285.
Drainakis, G., Pantazopoulos, P., Katsaros, K.V., Sourlas, V., Amditis, A., Kaklamani, D.I., 2023. From centralized to federated learning: Exploring performance and end-to-end resource consumption. Comput. Netw. 225, 109657.
Drisko, J.W., Maschi, T., 2016. Content analysis. Oxford University Press, USA.
El Balbali, H., Abou El Kalam, A., 2023. AI-driven big data quality improvement for efficient threat detection in agricultural IoT systems. In: AI2SD. Springer, pp. 39–47.
Epperson, W., Gorantla, V., Moritz, D., Perer, A., 2023. Dead or alive: Continuous data profiling for interactive data science. IEEE Trans. Vis. Comput. Graphics.
Fan, D., Wu, Y., Li, X., 2021. On the fairness of swarm learning in skin lesion classification. In: MICCAI. Springer, pp. 120–129.
Felderer, M., Ramler, R., 2021. Quality assurance for AI-based systems: Overview and challenges (introduction to interactive session). In: SWQD. pp. 33–42.
Felderer, M., Russo, B., Auer, F., 2019. On testing data-intensive software systems. C-CPS.
Feroz, S.B., Sharmin, N., Sevas, M.S., 2024. An empirical analysis of hyperparameter tuning impact on ensemble machine learning algorithm for earthquake damage prediction. Asian J. Civ. Eng. 1–27.
Ferrara, C., Sellitto, G., Ferrucci, F., Palomba, F., De Lucia, A., 2024. Fairness-aware machine learning engineering: how far are we? ESE 29 (1), 9.

Figueroa, C., Knowles, T., Kukreja, V., Lung, C.-H., 2023. IoT management with container orchestration. In: ICEIB. IEEE, pp. 49–54.
Franch, X., Martínez-Fernández, S., Ayala, C.P., Gómez, C., 2022. Architectural decisions in AI-based systems: An ontological view. In: QUATIC. pp. 18–27.
Gao, X., Zhai, J., Ma, S., Shen, C., Chen, Y., Wang, Q., 2022. FairNeuron: improving deep neural network fairness with adversary games on selective neurons. In: ICSE. pp. 921–933.
Garbuk, S.V., 2018. Intellimetry as a way to ensure AI trustworthiness. In: IC-AIAI. pp. 27–30.
Gaspar, D., Silva, P., Silva, C., 2024. Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron. ICE/ ITMC.
Gawhade, R., Bohara, L.R., Mathew, J., Bari, P., 2022. Computerized data-preprocessing to improve data quality. In: ICPC2T. pp. 1–6.
Geyer, P., Singaravel, S., 2018. Component-based machine learning for performance prediction in building design. Appl. Energy 228, 1439–1453.
Gezici, B., Tarhan, A.K., 2022. Systematic literature review on software quality for AI-based software. ESE 27 (3), 66.
Ghai, B., Mueller, K., 2022. D-bias: A causality-based human-in-the-loop system for tackling algorithmic bias. TVCG 29 (1), 473–482.
Ghani, R., Rodolfa, K.T., Saleiro, P., Jesus, S., 2023. Addressing bias and fairness in machine learning: A practical guide and hands-on tutorial. In: SIGKDD. pp. 5779–5780.
Gómez-Carmona, O., Casado-Mansilla, D., López-de Ipiña, D., García-Zubia, J., 2024. Human-in-the-loop machine learning: Reconceptualizing the role of the user in interactive approaches. Internet Things 25, 101048.
Gongane, V.U., Munot, M.V., Anuse, A.D., 2024. A survey of explainable AI techniques for detection of fake news and hate speech on social media platforms. JCS 1–37.
González-Abad, J., López García, Á., Kozlov, V.Y., 2023. A container-based workflow for distributed training of deep learning algorithms in HPC clusters. Cl. Comp. 26 (5), 2815–2834.
Gu, X., Tianqing, Z., Li, J., Zhang, T., Ren, W., Choo, K.-K.R., 2022. Privacy, accuracy, and model fairness trade-offs in federated learning. Comput. Secur. 122, 102907.
Guijarro-Berdiñas, B., Fernandez-Lorenzo, S., Sánchez-Maroño, N., Fontenla-Romero, O., 2011. A privacy-preserving distributed and incremental learning method for intrusion detection. In: ICANN. Springer, pp. 415–421.
Gupta, G., Lakhwani, K., 2022. An enhanced approach to improve the encryption of big data using intelligent classification technique. Multimedia Tools Appl. 81 (18), 25171–25204.
Habibullah, K.M., Gay, G., Horkoff, J., 2023. Non-functional requirements for machine learning: Understanding current use and challenges among practitioners. RE.
Haindl, P., Hoch, T., Dominguez, J., Aperribai, J., Ure, N.K., Tunçel, M., 2022. Quality characteristics of a software platform for human- AI teaming in smart manufacturing. In: QUATIC. pp. 3–17.
Hassan, M.U., Rehmani, M.H., Kotagiri, R., Zhang, J., Chen, J., 2019. Differential privacy for renewable energy resources based smart metering. JPDC.
Heimerl, F., Koch, S., Bosch, H., Ertl, T., 2012. Visual classifier training for text document retrieval. TVCG 18 (12), 2839–2848.
Heisele, B., Ho, P., Poggio, T., 2011. Face recognition with support vector machines: Global versus component-based approach. In: ICCV. 2, IEEE, pp. 688–694.
Heyn, H.-M., Knauss, E., Pelliccione, P., 2023. A compositional approach to creating architecture frameworks with an application to distributed AI systems. JSS.
Hikal, N.A., Elgayar, M., 2020. Enhancing IoT botnets attack detection using machine learning-IDS and ensemble data preprocessing technique. In: ITAF. pp. 89–102.
Höhlein, K., Kern, M., Hewson, T., Westermann, R., 2020. A comparative study of convolutional neural network models for wind field downscaling. Meteorol. Appl. 27 (6), e1961.
Horkoff, J., 2019. Non-functional requirements for machine learning: Challenges and new directions. In: RE. pp. 386–391.
Hort, M., Chen, Z., Zhang, J.M., Harman, M., Sarro, F., 2023. Bias mitigation for machine learning classifiers: A comprehensive survey. JRC.
Hulten, G., 2018. Building intelligent systems: A guide to machine learning engineering. A Press.
Hussein, A.M., Idrees, A.K., Couturier, R., 2022. Distributed energy-efficient data reduction approach based on prediction and compression to reduce data transmission in IoT networks. Int. J. Commun. Syst. 35 (15), e5282.
Hutiri, W., Ding, A.Y., Kawsar, F., Mathur, A., 2023. Tiny, always-on, and fragile: Bias propagation through design choices in on-device machine learning workflows. TSEM 32 (6), 1–37.
International Organization for Standardization, 2001. ISO/IEC 9126:2001 Software engineering - Product quality. Tech. rep, ISO.
International Organization for Standardization, 2011a. ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation — System and software quality models. Tech. rep, ISO.
International Organization for Standardization, 2011b. ISO/IEC 42010:2011 - Systems and software engineering — Architecture description. Tech. rep, ISO.
International Organization for Standardization, 2023. ISO/IEC 25059:2023 Software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality model for AI systems. Tech. rep, ISO.
Ishikawa, F., Yoshioka, N., 2019. How do engineers perceive difficulties in engineering of machine-learning systems?-questionnaire survey. In: CESI. pp. 2–9.
Jagarlamudi, G.K., Yazdinejad, A., Parizi, R.M., Pouriyeh, S., 2023. Exploring privacy measurement in federated learning. J. Supercomput. 1–41.
Jain, D.K., Dutta, A.K., Verdú, E., Alsubai, S., Sait, A.R.W., 2023. An automated hyperparameter tuned deep learning model enabled facial emotion recognition for autonomous vehicle drivers. Image Vis. Comput. 133, 104659.
Jain, S., Kumar, P., 2023. Cost effective generic machine learning operation: A case study. In: ICDSNS. IEEE, pp. 1–6.
Jakubik, J., Vössing, M., Kühl, N., Walk, J., Satzger, G., 2024. Data-centric artificial intelligence. BISE 1–9.
Jena, S., Sundarrajan, S., Meena, A., Chandavarkar, B., 2022. Human-in-the-loop control and security for intelligent cyber-physical systems (CPSs) and IoT. In: ICDSIAI. Springer, pp. 393–403.
Jeong, H., Chung, T.-M., 2022. Security and privacy issues and solutions in federated learning for digital healthcare. In: FDSE. Springer, pp. 316–331.
Jones, M.L., Kaufman, E., Edenberg, E., 2018. AI and the ethics of automating consent. SP 16 (3), 64–72.
Joraviya, N., Gohil, B.N., Rao, U.P., 2024. DL-HIDS: deep learning-based host intrusion detection system using system calls-to-image for containerized cloud environment. J. Supercomput. 1–29.
Joshi, S., Hasan, B., Brindha, R., 2024. Optimal declarative orchestration of full lifecycle of machine learning models for cloud native. In: ICAAIC. IEEE, pp. 578–582.
Kalananthan, S., Kichutkin, A., Shang, Z., Strausz, A., Bautiste, F.J.S., El-Assady, M., 2023. MindSet: A bias-detection interface using a visual human-in-the-loop workflow. In: ECAI. Springer, pp. 93–105.
Kang, Y., Chiu, Y.-W., Lin, M.-Y., Su, F.-Y., Huang, S.-T., 2021. Towards model-informed precision dosing with expert-in-the-loop machine learning. In: IRI. IEEE, pp. 342–347.
Kantarcioglu, M., Shaon, F., 2019. Securing big data in the age of AI. In: TPS-ISA. IEEE, pp. 218–220.
Kästner, C., Kang, E., 2020. Teaching software engineering for AI-enabled systems. In: ICSE. pp. 45–48.
Kaur, H., Rani, V., Kumar, M., Sachdeva, M., Mittal, A., Kumar, K., 2023. Federated learning: a comprehensive review of recent advances and applications. Multimedia Tools Appl. 1–24.
Kerschke, P., Hoos, H.H., Neumann, F., Trautmann, H., 2019. Automated algorithm selection: Survey and perspectives. Evol. Comput. 27 (1), 3–45.
Khan, S., Tsutsumi, S., Yairi, T., Nakasuka, S., 2021. Robustness of AI-based prognostic and systems health management. Annu. Rev. Control..
Kim, M., Günlü, O., Schaefer, R.F., 2021. Federated learning with local differential privacy: Trade-offs between privacy, utility, and communication. In: ICASSP. IEEE, pp. 2650–2654.
Kirienko, M., Sollini, M., Ninatti, G., Loiacono, D., Giacomello, E., Gozzi, N., Amigoni, F., Mainardi, L., Lanzi, P.L., Chiti, A., 2021. Distributed learning: a reliable privacy-preserving strategy to change multicenter collaborations using AI. EJNMMI 48, 3791–3804.
Kitchenham, B., Charters, S., 2007. Guidelines for performing Systematic Literature Reviews in Software Engineering. Tech. rep, EBSE Technical Report.
Kolltveit, A.B., Li, J., 2022. Operationalizing machine learning models: A systematic literature review. In: SE4RAI. pp. 1–8.
Kröll, M., Burova-Keßler, K., 2021. AI and learning in the context of digital transformation. In: AHFE. Springer, pp. 36–43.
Kumarakulasinghe, N.B., Blomberg, T., Liu, J., Leao, A.S., Papapetrou, P., 2020. Evaluating local interpretable model-agnostic explanations on clinical machine learning classification models. In: CBMS. IEEE, pp. 7–12.
Kunang, Y.N., Nurmaini, S., Stiawan, D., Suprapto, B.Y., 2021. Attack classification of an intrusion detection system using deep learning and hyperparameter optimization. JISA 58, 102804.
Kuwajima, H., Ishikawa, F., 2019. Adapting SQuaRE for quality assessment of artificial intelligence systems. In: ISSREW. pp. 13–18.
Kuwajima, H., Yasuoka, H., Nakae, T., 2020. Engineering problems in machine learning systems. ML.
Lampe, B., Meng, W., 2023. A survey of deep learning-based intrusion detection in automotive applications. Expert Syst. Appl. 221, 119771.
Lane, T., Brodley, C.E., 2019. Temporal sequence learning and data reduction for anomaly detection. TISSEC 2 (3), 295–331.
Laqtib, S., Yassini, K.E., Hasnaoui, M.L., 2019. A deep learning methods for intrusion detection systems based machine learning in MANET. In: SCA. pp. 1–8.
Lee, M.S.A., Singh, J., 2021. The landscape and gaps in open source fairness toolkits. In: CHI. pp. 1–13.
Lewis, G.A., Ozkaya, I., Xu, X., 2021. Software architecture challenges for ML systems. In: ICSME. IEEE, pp. 634–638.
Liao, L., Li, H., Shang, W., Ma, L., 2022. An empirical study of the impact of hyperparameter tuning and model optimization on the performance properties of deep neural networks. TOSEM 31 (3), 1–40.
Liu, J., 2022. Human-in-the-loop ethical AI for care robots and confucian virtue ethics. In: ICSR. Springer, pp. 674–688.
Liu, Q., Chen, L., Jiang, H., Wu, J., Wang, T., Peng, T., Wang, G., 2022a. A collaborative deep learning microservice for backdoor defenses in industrial IoT networks. Ad Hoc Netw. 124, 102727.

Liu, H., Eksmo, S., Risberg, J., Hebig, R., 2020. Emerging and changing tasks in the development process for machine learning systems. In: ICSSP. pp. 125–134.

Liu, L., Yu, J., Ding, Z., 2022b. Adaptive and efficient GPU time sharing for hyperparameter tuning in cloud. In: ICPP. pp. 1–11.

Logothetis, I., Barnett, S., Hoon, L., Thudumu, S., Mathew, J., Luckhoff, C., O'Reilly, G., Collard, D., Vasa, R., Mouzakis, K., et al., 2022. Pims: A pre-ML labelling tool. In: E-Science. IEEE, pp. 431–432.

Love, P.E., Fang, W., Matthews, J., Porter, S., Luo, H., Ding, L., 2023. Explainable artificial intelligence (XAI): Precepts, models, and opportunities for research in construction. AEI 57, 102024.

Lundberg, L., Bosch, J., Häggander, D., Bengtsson, P.-O., 1999. Quality attributes in software architecture design. In: IASTED. pp. 353–362.

Lwakatare, L.E., Raj, A., Crnkovic, I., Bosch, J., Olsson, H.H., 2020. Large-scale machine learning systems in real-world industrial settings: A review of challenges and solutions. Inf. Softw. Technol ..

Lycklama, H., Burkhalter, L., Viand, A., Küchler, N., Hithnawi, A., 2023. RoFL: Robustness of secure federated learning. In: SP. IEEE, pp. 453–476.

Lyu, L., Yu, H., Ma, X., Chen, C., Sun, L., Zhao, J., Yang, Q., Philip, S.Y., 2022. Privacy and robustness in federated learning: Attacks and defenses. IEEE Trans. Neural Networks Learn. Syst..

Maan, J., 2022. Deep learning-driven explainable AI using generative adversarial network (GAN). In: INDICON. IEEE, pp. 1–5.

Mandela, N., Alam, I., Amudha, A., Priyanka, D., Singh, D.K., et al., 2023. Enabling scalable applications with intelligent distributed data processing. In: INCOFT. IEEE, pp. 1–7.

McGraw, G., 2020. Security engineering for machine learning (keynote). In: SIGSOFT. pp. 2–2.

Miceli, M., Posada, J., Yang, T., 2022. Studying up machine learning data: Why talk about bias when we mean power? Human- Comput. Interact. 6 (GROUP), 1–14.

Minh, D., Wang, H.X., Li, Y.F., Nguyen, T.N., 2022. Explainable artificial intelligence: a comprehensive review. Artif. Intell. Rev. 1–66.

Mohan, V., Othmane, L.B., 2016. Secdevops: Is it a marketing buzzword?-mapping research on security in devops. In: 2016 11th International Conference on Availability, Reliability and Security. ARES, IEEE, pp. 542–547.

Monson-Haefel, R., 2009. 97 things every software architect should know: collective wisdom from the experts. O'Reilly Media, Inc..

Moraffah, R., Karami, M., Guo, R., Raglin, A., Liu, H., 2020. Causal interpretability for machine learning-problems, methods and evaluation. SIGKDD 22 (1), 18–33.

Mori, T., Uchihira, N., 2019. Balancing the trade-off between accuracy and interpretability in software defect prediction. ESE 24, 779–825.

Morovati, M.M., Nikanjam, A., Khomh, F., Jiang, Z.M., 2023. Bugs in machine learning-based systems: a faultload benchmark. ESE.

Mostafa, F., Tao, L., Yu, W., 2021. An effective architecture of digital twin system to support human decision making and AI-driven autonomy. CCPE 33 (19), e6111.

Muccini, H., Vaidhyanathan, K., 2021. Software architecture for ML-based systems: What exists and what lies ahead. In: WAIN. pp. 121–128.

Nakamichi, K., Ohashi, K., Namba, I., Yamamoto, R., Aoyama, M., Joeckel, L., Siebert, J., Heidrich, J., 2020. Requirements-driven method to determine quality characteristics and measurements for machine learning software and its evaluation. In: RE. pp. 260–270.

Nassef, O., Sun, W., Purmehdi, H., Tatipamula, M., Mahmoodi, T., 2022. A survey: Distributed machine learning for 5G and beyond. Comput. Netw. 207, 108820.

Naydenov, N., Ruseva, S., 2022. Combining container orchestration and machine learning in the cloud: A systematic mapping study. In: INFOTEH. IEEE, pp. 1–6.

Nguyen, T.N., Choo, R., 2021. Human-in-the-loop XAI-enabled vulnerability detection, investigation, and mitigation. In: ASE. IEEE, pp. 1210–1212.

Noureen, S., Zubair, M., Ali, M., Mehmood, M.Q., 2021. Deep learning based sequence modeling for optical response retrieval of photonic nanostructures. In: IBCAST. IEEE, pp. 289–292.

Obaid, H.S., Dheyab, S.A., Sabry, S.S., 2019. The impact of data pre-processing techniques and dimensionality reduction on the accuracy of machine learning. In: IEMECON. IEEE, pp. 279–283.

Openja, M., Majidi, F., Khomh, F., Chembakottu, B., Li, H., 2022. Studying the practices of deploying machine learning projects on docker. In: EASE. pp. 190–200.

Oppold, S., Herschel, M., 2020. A system framework for personalized and transparent data-driven decisions. In: CAiSE. Springer, pp. 153–168.

Ottoni, A.L.C., Souza, A.M., Novo, M.S., 2023. Automated hyperparameter tuning for crack lee2021landscapeimage classification with deep learning. Soft Comput. 27 (23), 18383–18402.

Ozkaya, I., 2020. What is really different in engineering AI-enabled systems? IEEE Softw..

Pansara, R.R., Kasula, B.Y., Bhatia, A.B., Whig, P., 2024. Enhancing sustainable development through machine learning-driven master data management. In: International Conference on Sustainable Development Through Machine Learning, AI and IoT. Springer, pp. 332–341.

Pawlicki, M., Pawlicka, A., Śrutek, M., Kozik, R., Choraś, M., 2023. Interpreting intrusions-the role of explainability in AI-based intrusion detection systems. In: CORES. Springer, pp. 45–53.

Peldszus, S., Knopp, H., Sens, Y., Berger, T., 2023. Towards ML-integration and training patterns for AI-enabled systems. In: International Conference on Bridging the Gap Between AI and Reality. Springer, pp. 434–452.

Perera, A., Aleti, A., Tantithamthavorn, C., Jiarpakdee, J., Turhan, B., Kuhn, L., Walker, K., 2022. Search-based fairness testing for regression-based machine learning systems. ESE.

Perrone, V., Donini, M., Zafar, M.B., Schmucker, R., Kenthapadi, K., Archambeau, C., 2021. Fair bayesian optimization. In: AIES. pp. 854–863.

Petrelli, D., Dadzie, A.-S., Lanfranchi, V., 2012. Mediating between AI and highly specialized users. AI Mag. 30 (4), 95.

Picard, S., Chapdelaine, C., Cappi, C., Gardes, L., Jenn, E., Lefèvre, B., Soumarmon, T., 2020. Ensuring dataset quality for machine learning certification. In: ISSREW. pp. 275–282.

Pise, N., Kulkarni, P., 2016. Algorithm selection for classification problems. In: SAI. IEEE, pp. 203–211.

Poth, A., Meyer, B., Schlicht, P., Riel, A., 2020. Quality assurance for machine learning–an approach to function and system safeguarding. In: QRS. pp. 22–29.

Priestley, M., O'donnell, F., Simperl, E., 2023. A survey of data quality requirements that matter in ML development pipelines. JDIQ 15 (2), 1–39.

Qu, F., Zhang, J., Shao, Z., Qi, S., 2017. An intrusion detection model based on deep belief network. In: ICNCC. pp. 97–101.

Rajapaksha, D., Bergmeir, C., Buntine, W., 2020. LoRMIkA: Local rule-based model interpretability with k-optimal associations. Inform. Sci. 540, 221–241.

Rajendran, P.T., Espinoza, H., Delaborde, A., Mraidha, C., 2021. Human-in-the-loop learning methods toward safe DL-based autonomous systems: A review. In: SAFECOMP. Springer, pp. 251–264.

Ramírez-Gallego, S., Krawczyk, B., García, S., Woźniak, M., Herrera, F., 2017. A survey on data preprocessing for data stream mining: Current status and future directions. Neurocomputing 239, 39–57.

Ramkumar, M., Malathi, K., Pavithra, K., 2023. Optimizing machine learning model accuracy via OBNT algorithm: Advanced data preprocessing technique. In: ICSES. IEEE, pp. 1–6.

Rao, J., Bu, X., Wang, K., Xu, C.-Z., 2011. Self-adaptive provisioning of virtualized resources in cloud computing. In: SIGMETRICS. pp. 129–130.

Rashid, S.M.M., Toufikuzzaman, M., Hossain, M.S., 2023. A deep learning based semi-supervised network intrusion detection system robust to adversarial attacks. In: NSysS. pp. 25–34.

Rashidi, H.H., Tran, N., Albahra, S., Dang, L.T., 2021. Machine learning in health care and laboratory medicine: General overview of supervised learning and Auto-ML. Int. J. Lab. Hematol. 43, 15–22.

Reed, S.K., 2016. A taxonomic analysis of abstraction. Perspect. Psychol. Sci. 11 (6), 817–837.

Rendleman, M.C., Buatti, J.M., Braun, T.A., Smith, B.J., Nwakama, C., Beichel, R.R., Brown, B., Casavant, T.L., 2019. Machine learning with the TCGA-HNSC dataset: improving usability by addressing inconsistency, sparsity, and high-dimensionality. BMC Bioinformatics 20, 1–9.

Rey, M.I., Galende, M., Fuente, M.J., Sainz-Palmero, G., 2017. Multi-objective based fuzzy rule based systems (FRBSs) for trade-off improvement in accuracy and interpretability: A rule relevance point of view. KBS 127, 67–84.

Rimal, Y., Sharma, N., Alsadoon, A., 2024. The accuracy of machine learning models relies on hyperparameter tuning: student result classification using random forest, randomized search, grid search, bayesian, genetic, and optuna algorithms. Multimedia Tools Appl. 1–16.

Rodríguez, D.M., Cuéllar, M.P., Morales, D.P., 2024. Concept logic trees: enabling user interaction for transparent image classification and human-in-the-loop learning. AI 1–13.

Romsaiyud, W., Schnoor, H., Hasselbring, W., 2019. Improving k-nearest neighbor pattern recognition models for privacy-preserving data analysis. In: Big Data. IEEE, pp. 5804–5813.

Rovnyagin, M.M., Hrapov, A.S., Guminskaia, A.V., Orlov, A.P., 2020. ML-based heterogeneous container orchestration architecture. In: EIConRus. IEEE, pp. 477–481.

Rovnyagin, M.M., Timofeev, K.V., Elenkin, A.A., Shipugin, V.A., 2019. Cloud computing architecture for high-volume ML-based solutions. In: EIConRus. IEEE, pp. 315–318.

Saadatfar, H., Kiani-Zadegan, Z., Ghahremani-Nezhad, B., 2024. US-LIME: Increasing fidelity in LIME using uncertainty sampling on tabular data. Neurocomputing 597, 127969.

Sajid, S., von Zernichow, B.M., Soylu, A., Roman, D., 2019. Predictive data transformation suggestions in grafterizer using machine learning. In: MTSR. Springer, pp. 137–149.

Sanju, P., 2023. Enhancing intrusion detection in IoT systems: A hybrid metaheuristics-deep learning approach with ensemble of recurrent neural networks. JER 11 (4), 356–361.

Santhanam, P., 2020. Quality management of machine learning systems. In: EDSMLS. pp. 1–13.

Santos, L., Ferreira, L., 2023. Atlantic—Automated data preprocessing framework for supervised machine learning. Softw. Impacts 17, 100532.

Sarjoughian, H.S., Fallah, F., Saeidi, S., Yellig, E.J., 2023. Transforming discrete event models to machine learning models. In: WSC. IEEE, pp. 2662–2673.

Sattler, F., Müller, K.-R., Wiegand, T., Samek, W., 2020. On the byzantine robustness of clustered federated learning. In: ICASSP. IEEE, pp. 8861–8865.

Seo, E., Kim, H., Chung, T.-M., 2019. Profiling-based classification algorithms for security applications in Internet of Things. In: ICIOT. IEEE, pp. 138–146.

Serban, A., van der Blom, K., Hoos, H., Visser, J., 2020. Adoption and effects of software engineering best practices in machine learning. In: ESEM. pp. 1–12.
Serban, A., Visser, J., 2022. Adapting software architectures to machine learning challenges. In: SANER. pp. 152–163.
Shadab, N., Salado, A., 2020. Towards an interface description template for reusing AI-enabled systems. In: SMC. IEEE, pp. 2893–2900.
Shahoud, S., Winter, M., Khalloof, H., Duepmeier, C., Hagenmeyer, V., 2021. An extended meta learning approach for automating model selection in big data environments using microservice and container virtualizationz technologies. IoT 16, 100432.
Shand, C., Fong, R., Butt, U., 2023. How explainable artificial intelligence (XAI) models can be used within intrusion detection systems (IDS) to enhance an analyst's trust and understanding. In: ICGS3. Springer, pp. 321–342.
Shen, S., Zhu, T., Wu, D., Wang, W., Zhou, W., 2022. From distributed machine learning to federated learning: In the view of data privacy and security. CCPE 34 (16), e6002.
Shi, S., Wang, Q., Chu, X., Li, B., Qin, Y., Liu, R., Zhao, X., 2020. Communication-efficient distributed deep learning with merged gradient sparsification on GPUs. In: INFOCOM. IEEE, pp. 406–415.
Shin, S., Boyapati, M., Suo, K., Kang, K., Son, J., 2023. An empirical analysis of image augmentation against model inversion attack in federated learning. Clust. Comput. 26 (1), 349–366.
Shivashankar, K., Martini, A., 2022. Maintainability challenges in ML: A systematic literature review. In: SEAA. IEEE, pp. 60–67.
Siddiqi, S., Kern, R., Boehm, M., 2023. SAGA: A scalable framework for optimizing data cleaning pipelines for machine learning applications. MoD 1 (3), 1–26.
Siddiqui, H., Khendek, F., Toeroe, M., 2023. Microservices based architectures for IoT systems-state-of-the-art review. IoT 100854.
Siebert, J., Joeckel, L., Heidrich, J., Trendowicz, A., Nakamichi, K., Ohashi, K., Namba, I., Yamamoto, R., Aoyama, M., 2022. Construction of a quality model for machine learning systems. SQ.
Singaravel, S., Suykens, J., Geyer, P., 2018. Deep-learning neural-network architectures and methods: Using component-based models in building-design energy prediction. Adv. Eng. Inf..
Singh, A.P., Chaudhari, S., 2020. Embedded machine learning-based data reduction in application-specific constrained IoT networks. In: SIGAPP. pp. 747–753.
Smith, A.L., Clifford, R., 2020. Quality characteristics of artificially intelligent systems. In: IWESQ APSEC. pp. 1–6.
Soares, E., Angelov, P.P., Costa, B., Castro, M.P.G., Nageshrao, S., Filev, D., 2020. Explaining deep learning models through rule-based approximation and visualization. TFS.
Soui, M., Gasmi, I., Smiti, S., Ghédira, K., 2019. Rule-based credit risk assessment model using multi-objective evolutionary algorithms. Expert Syst. Appl. 126, 144–157.
Sperrle, F., El-Assady, M., Guo, G., Borgo, R., Chau, D.H., Endert, A., Keim, D., 2021. A survey of human-centered evaluations in human-centered machine learning. In: Computer Graphics Forum, vol. 40. Wiley Online Library, pp. 543–568.
Staron, M., 2020. Action research in software engineering. Springer.
Sun, Y., Haghighat, F., Fung, B.C., 2022. Trade-off between accuracy and fairness of data-driven building and indoor environment models: A comparative study of pre-processing methods. Energy.
Tao, C., Gao, J., Wang, T., 2019. Testing and quality validation for AI software–perspectives, issues, and practices. JSS.
Tomei, M., Schwing, A., Narayanasamy, S., Kumar, R., 2019. Sensor training data reduction for autonomous vehicles. In: MOBICOM. pp. 45–50.
Trajanov, R., Dimeski, S., Popovski, M., Korošec, P., Eftimov, T., 2022. Explainable landscape analysis in automated algorithm performance prediction. In: EvoStar. Springer, pp. 207–222.
Truong, H.-L., 2023. Coordination-aware assurance for end-to-end machine learning systems: the R3E approach. In: AI Assurance. pp. 339–367.
Tsimenidis, S., Lagkas, T., Rantos, K., 2022. Deep learning in IoT intrusion detection. J. Netw. Syst. Manage. 30 (1), 8.
Tuladhar, A., Rajashekar, D., Forkert, N.D., 2023. Distributed learning in healthcare. Trends Artif. Intell. Big Data E- Heal. 183–212.
Tverdal, S., Goknil, A., Nguyen, P., Husom, E.J., Sen, S., Ruh, J., Flamigni, F., 2023. Edge-based data profiling and repair as a service for IoT. In: IoT. pp. 17–24.
ur Rehman, M.H., Liew, C.S., Abbas, A., Jayaraman, P.P., Wah, T.Y., Khan, S.U., 2016. Big data reduction methods: a survey. Data Sci. Eng. 1, 265–284.
Van Der Weide, T., Papadopoulos, D., Smirnov, O., Zielinski, M., Van Kasteren, T., 2017. Versioning for end-to-end machine learning pipelines. In: DM-ML. pp. 1–9.
Vieira, C.P., Digiampietri, L.A., 2022. Machine learning post-hoc interpretability: a systematic mapping study. In: SBSI. ACM, pp. 1–8.
Vogelsang, A., Borg, M., 2019. Requirements engineering for machine learning: Perspectives from data scientists. In: REW. pp. 245–251.
Vojíř, S., Kliegr, T., 2020. Editable machine learning models? A rule-based framework for user studies of explainability. ADAC.
Wan, Z., Xia, X., Lo, D., Murphy, G.C., 2019. How does machine learning change software development practices? TSE.
Wang, C.-J., Li, P.-P., Zhou, X.-Y., Liu, N., 2021. Privacy-preserving breast cancer prediction via inner-product functional encryption. In: ICCC. IEEE, pp. 539–543.
Wang, X., Miao, M., 2022. A framework for requirements specification of machine-learning systems. In: SEKE. pp. 7–12.
Wanganoo, L., Shukla, V.K., 2020. Real-time data monitoring in cold supply chain through NB-IoT. In: ICCCNT. pp. 1–6.
Warnett, S.J., Zdun, U., 2022. Architectural design decisions for machine learning deployment. In: ICSA. IEEE, pp. 90–100.
Washizaki, H., Uchida, H., Khomh, F., Guéhéneuc, Y.-G., 2019. Studying software engineering patterns for designing machine learning systems. In: IWESEP. pp. 49–495.
Weng, D., 2023. Performance and energy evaluation of lightweight cryptography for small IoT devices. In: UEMCON. IEEE, pp. 289–295.
Winter, M., Jackson, P., Fallahkhair, S., 2023. Gesture Me: A machine learning tool for designers to train gesture classifiers. In: CHIRA. Springer, pp. 336–352.
Wonsil, J., Sullivan, J., Seltzer, M., Pocock, A., 2023. Integrated reproducibility with self-describing machine learning models. In: CRR. pp. 1–14.
Wu, Y.-H., Huang, X.-H., Liu, J.-X., Chang, L., 2022a. A big data encryption method based on lorenz and feistel structures. In: ICCEAI. IEEE, pp. 1–5.
Wu, L., Perin, G., Picek, S., 2022b. I choose you: Automated hyperparameter tuning for deep learning-based side-channel analysis. Trans. Emerg. Top. Comput..
Xin, D., Ma, L., Liu, J., Macke, S., Song, S., Parameswaran, A., 2018. Accelerating human-in-the-loop machine learning: Challenges and opportunities. In: DEEM. pp. 1–4.
Yap, R.H., 2021. Towards certifying trustworthy machine learning systems. In: TAILOR. pp. 77–82.
Yokoyama, H., 2019. Machine learning system architectural pattern for improving operational stability. In: ICSA. pp. 267–274.
Yousefi, M.H.N., Degeler, V., Lazovik, A., 2023. Empowering machine learning development with service-oriented computing principles. In: SummerSOC. Springer, pp. 24–44.
Zelaya, C.V.G., 2019. Towards explaining the effects of data preprocessing on machine learning. In: ICDE. pp. 2086–2090.
Zerka, F., Urovi, V., Bottari, F., Leijenaar, R.T., Walsh, S., Gabrani-Juma, H., Gueuning, M., Vaidyanathan, A., Vos, W., Occhipinti, M., et al., 2021. Privacy preserving distributed learning classifiers–sequential learning with small sets of data. Comput. Biol. Med. 136, 104716.
Zhang, N., Bahsoon, R., Tziritas, N., Theodoropoulos, G., 2022. Explainable human-in-the-loop dynamic data-driven digital twins. In: DDDAS. Springer, pp. 233–243.
Zhang, J.M., Harman, M., Ma, L., Liu, Y., 2020. Machine learning testing: Survey, landscapes and horizons. TSE.
Zhang, J., Shu, Y., Yu, H., 2023. Fairness in design: A framework for facilitating ethical artificial intelligence designs. IJCS 7 (1), 32–39.
Zhang, C., Xie, Y., Bai, H., Yu, B., Li, W., Gao, Y., 2021. A survey on federated learning. KBS 216, 106775.
Zhao, B., Song, M., Liu, S., Sun, L., Jiang, W., Qian, H., Zhang, X.-Y., Zhang, Y., Jiang, T., 2023. MosaicNet: A deep-learning-based multi-tile biomedical image stitching method. In: EMBC. IEEE, pp. 1–4.
Zhou, J., Su, Z., Ni, J., Wang, Y., Pan, Y., Xing, R., 2022. Personalized privacy-preserving federated learning: Optimized trade-off between utility and privacy. In: GLOBECOM. IEEE, pp. 4872–4877.